Neural Network (NN)

Lương Thái Lê

Outline

1. NN Introduction
2. Components of a NN
3. ANN architect
4. Perceptron
5. Gradient descent
6. Multi-layer ANN & Back propagation

Artificial Neural Network – Introduction (1)

- Artificial Neural Network (ANN)
  - Simulation of biological neural systems (human brains)
  - Is a structure made of a number of interconnected neurons.
- Each neuron:
  - has an Input/Output characteristics
  - Perform a local computation
- The output value of a neuron is determined by:
  - it’s Input/Output characteristics
  - Its connections with other neurons
  - additional inputs (if any)

Artificial Neural Network – Introduction (2)

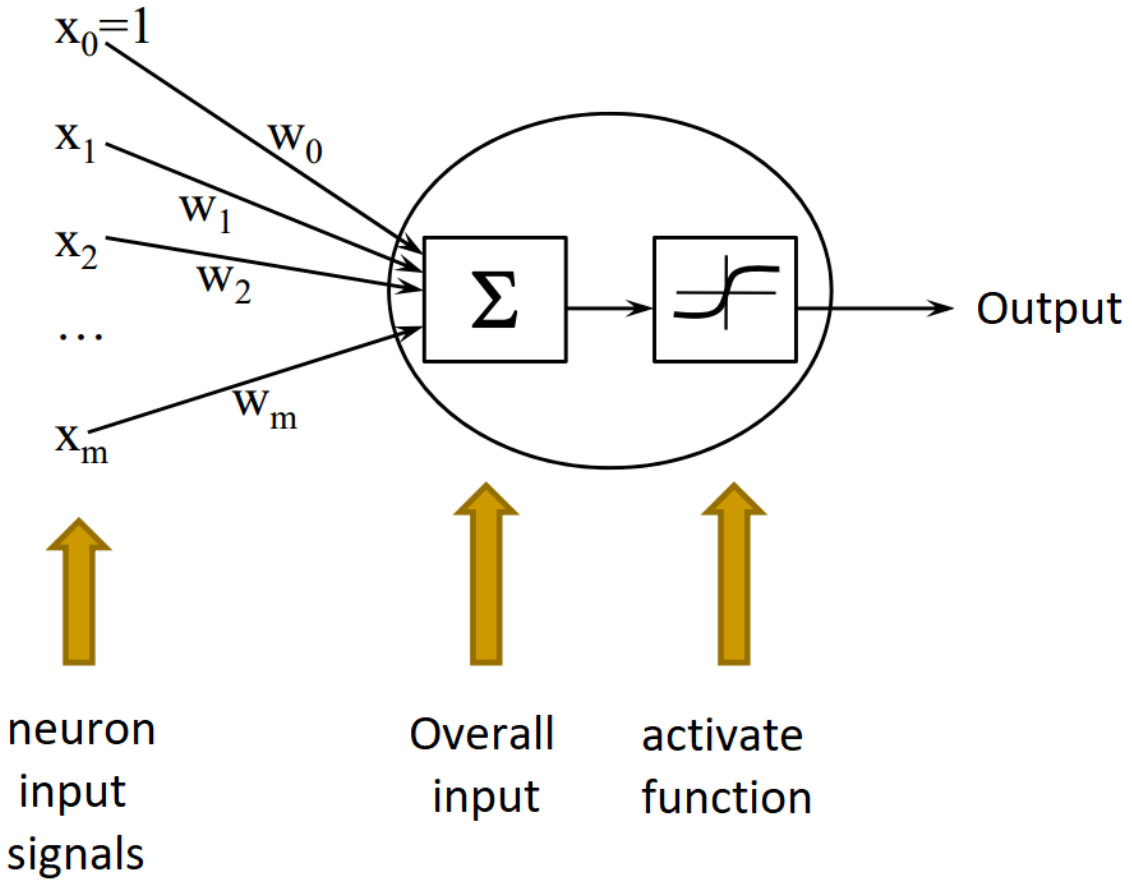
- ANN can be viewed as a structure that processes information in a distributed and highly parallel method.
- ANN has the ability to learn, recall, and generalize from learning data – by assigning and adjusting (adapting) the weight values of connections between neurons
- The objective function of an ANN is determined by:
  - The architecture (topology) of the neural network
  - Input/output characteristics of each neuron
  - Training Strategies
  - Training data

ANN – Typical applications

- Image processing and Computer vision
  - Matching, preprocessing, image segmentation and analysis, computer vision, image compression, processing and understanding images that change over time
- Signal processing
  - Signal and morphological seismic analysis, earthquake
- Pattern recognition
  - Attribute extraction, speech recognition and comprehension, fingerprint recognition, human face recognition
- Medical
  - Analyze and understand electrocardiographic signals, diagnose diseases, and process images in the medical field
- … and alots

Structure and operation of a neuron

- Input signal:
  - Each input signal x_i has a corresponding weight w_i
- w_0 is adjusted weights - bias (for $x_0 = 1$)
- Overall input:
  - is a function that integrates the input signals Net(**w**,**x**)
- Activate function:
  - calculate the output value of neuron

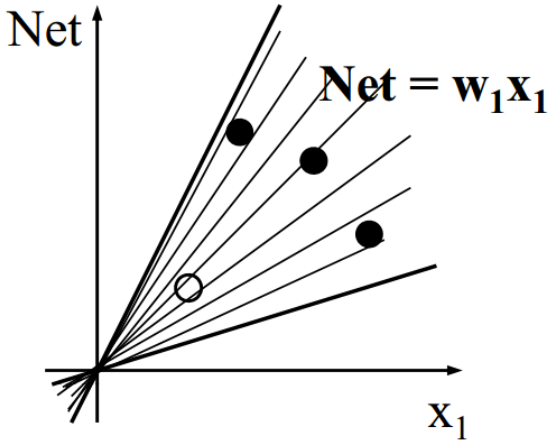


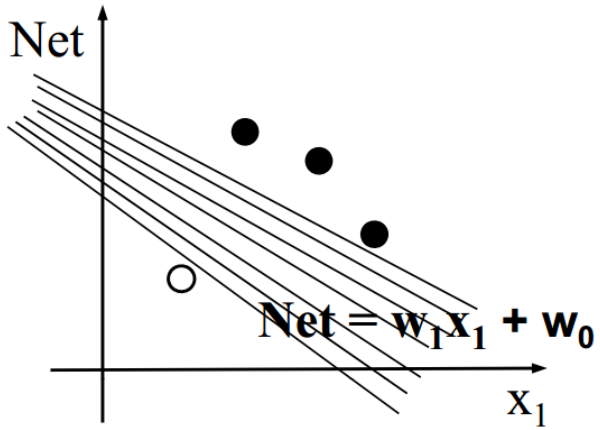
Overall input and adjustment

- The net input is usually calculated by a linear function:

$$Net = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m = \sum_{i=0}^{m} w_i x_i$$

- Meaning of bias:
  - The family of separation functions Net = $w_i x_i$ cannot separate examples into two classes
  - But Net = $w_i x_i + w_0$ could





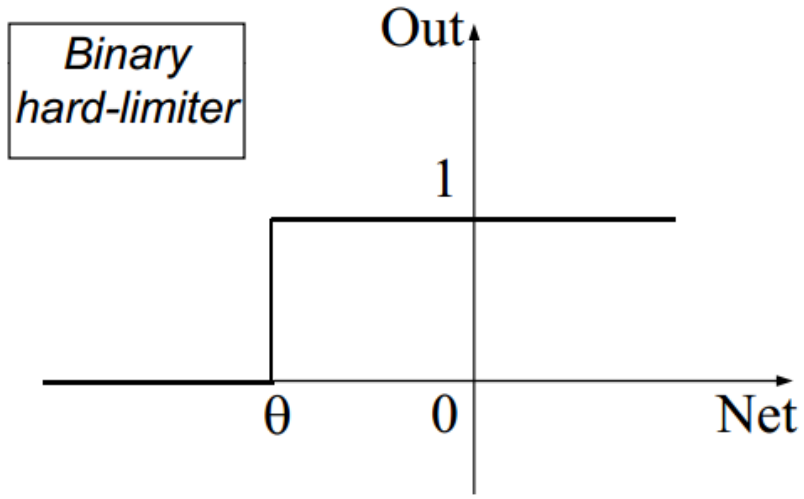
Activate function: Hard limiter

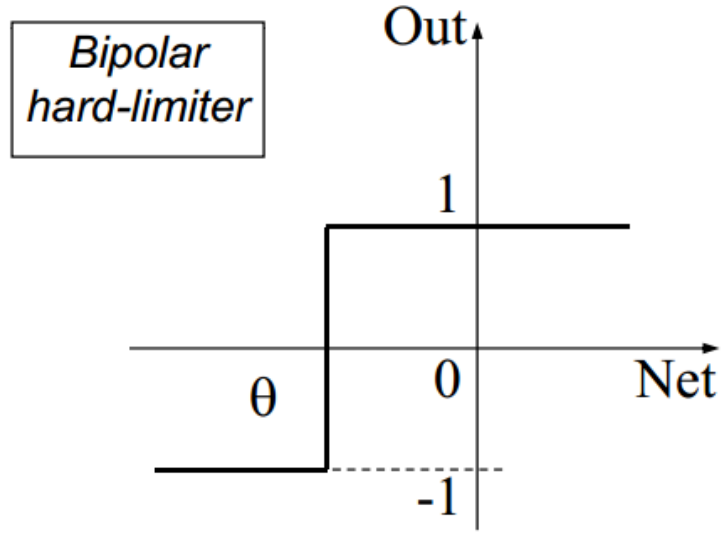
- Or call Threshold function:
- θ *is threshold value*
- Pitfall: not contituous and

the derivative is not continuous too

$$Out(Net) = hl1(Net, \theta) = \begin{cases} 1, & \text{if} \quad Net \geq \theta \\ 0, & \text{if other} \end{cases}$$

$$Out(Net) = hl2(Net, \theta) = sign(Net, \theta)$$

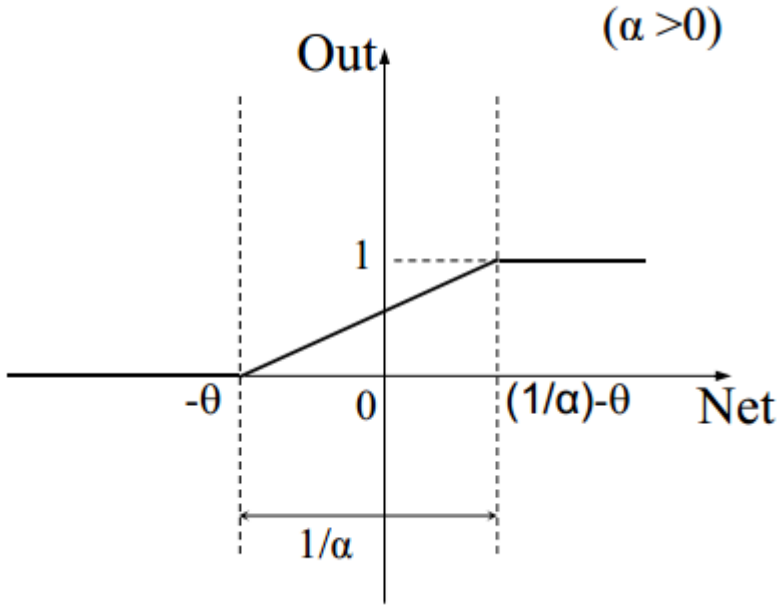




Activate function: Threshold logic

$$Out(Net) = tl(Net, \alpha, \theta) = \begin{cases} 0, & \text{if } Net < -\theta \\ \alpha(Net + \theta), & \text{if } -\theta \le Net \le \frac{1}{\alpha} - \theta \\ 1, & \text{if } Net > \frac{1}{\alpha} - \theta \end{cases}$$

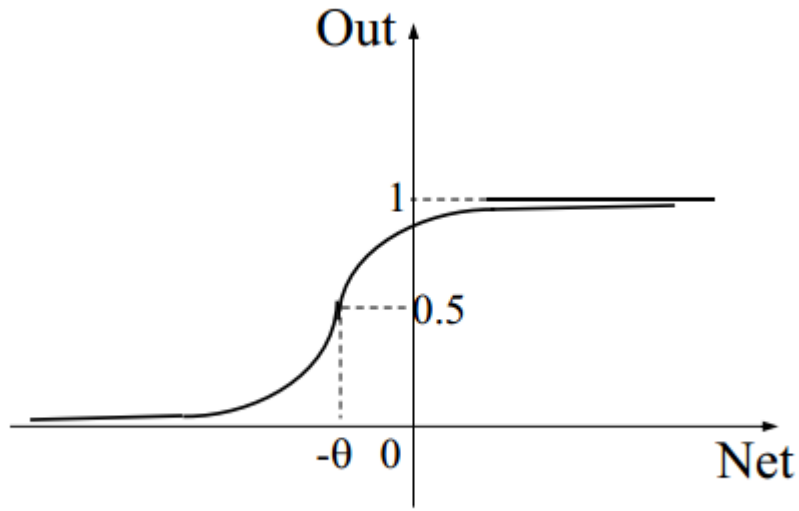
- α determine the slope of the linear interval
- Pitfall: Continuos but it's derivation is not



Activate function: Sigmoidal

$$Out(Net) = sf(Net, \alpha, \theta) = \frac{1}{1 + e^{-\alpha(Net+\theta)}}$$

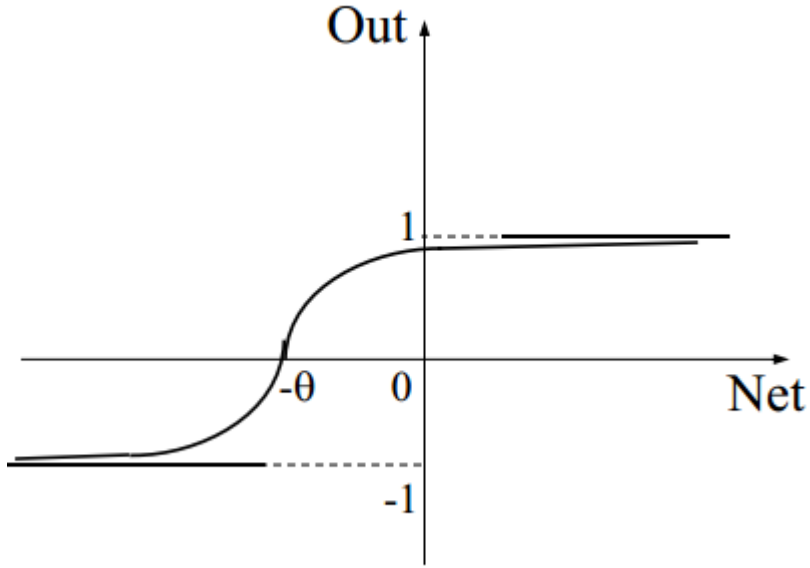
- the most commonly used
- α determine the slope of the linear interval
- Output is in (0,1)
- Advantage: Continuos and it's derivation is too



Activate function: Hyperbolic tangent

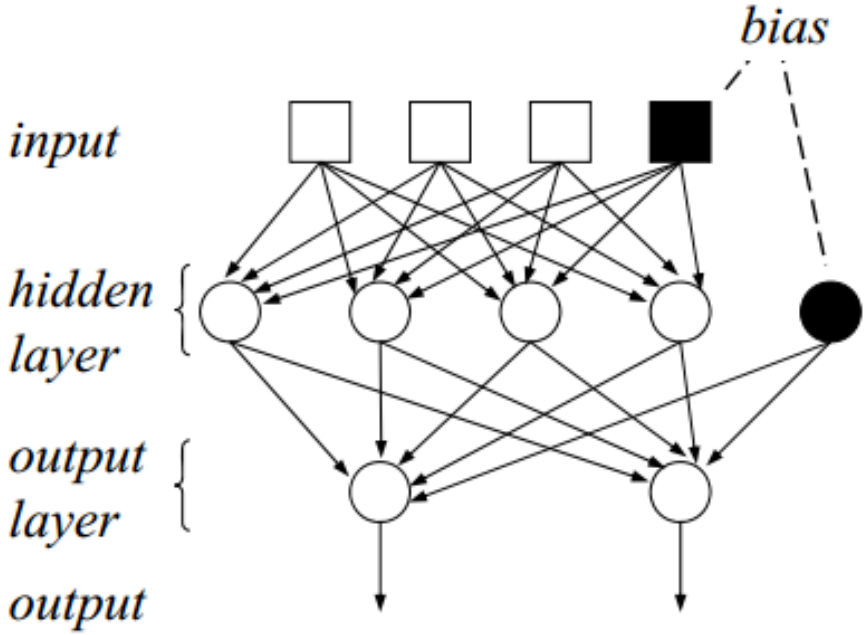
$$Out(Net) = \tanh(Net, \alpha, \theta) = \frac{1 - e^{-\alpha(Net+\theta)}}{1 + e^{-\alpha(Net+\theta)}} = \frac{2}{1 + e^{-\alpha(Net+\theta)}} - 1$$

- commonly used
- α determine the slope of the linear interval
- Output is in (-1,1)
- Advantage: Continuos and it's derivation is too



ANN – network architecture (1)

- The architecture of an ANN is determined by:
  - Number of input and output signals
  - Number of layers
  - Number of neurons on each layer
  - Number of weights for each neurons
  - How neurons (on a layer, or between layers) connect to each other
  - Which neurons receive error correction signals
- An ANN must has:
  - An input layer
  - An output layer
  - 0,1, or more hidden layer

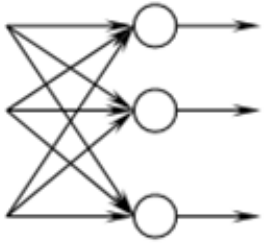


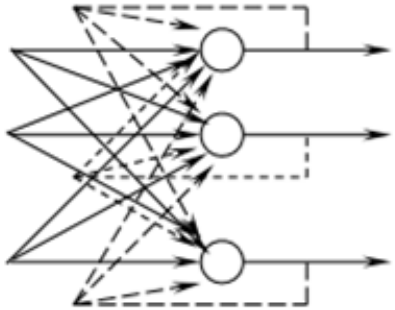
ANN – network architecture (2)

- An ANN is said to be fully connected if every output from one layer is connected to every neuron of the next layer
- An ANN is called a feedforward network if no output of one node is the input of another node of the same layer (or of a previous layer).
- When the outputs of a node link back to the inputs of a node of the same layer (or of a previous layer), it is a feedback network.
- Feedback networks that have closed loops are called recurrent networks

ANN – network architecture (3)

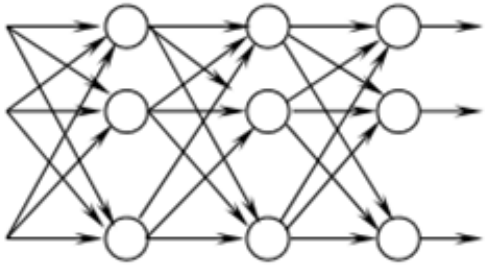
one-layer feedforward network

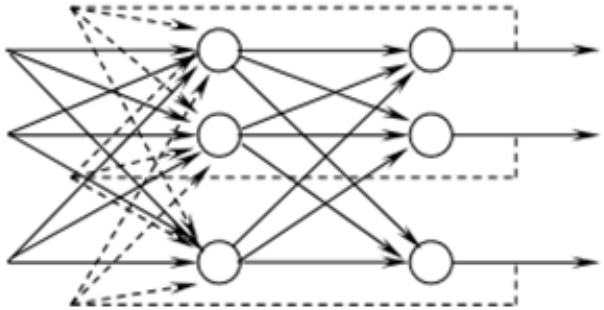




one-layer recurrent network

multi-layer feedforward network





multi-layer recurrent network

ANN – Learning rules

- Two types of learning in ANN:
  - *Parameter learning*
    The goal is to adaptively change the weights of the links in the neural network
  - *Structure learning*
    The goal is to adaptively change the network structure, including the number of neurons and the connection patterns between them
- These two types of learning can be done simultaneously or separately
- Most of the learning method in ANN are parameter learning

=> We will consider only parameter learning

Weight learning rules

- At learning step t, the degree of adjustment of the weight vector w is proportional to the product of the learning signal $r^{(t)}$ and input $\mathbf{x}^{(t)}$

$$\Delta \boldsymbol{w}^{(t)} = \eta r^{(t)} \boldsymbol{x}^{(t)}$$

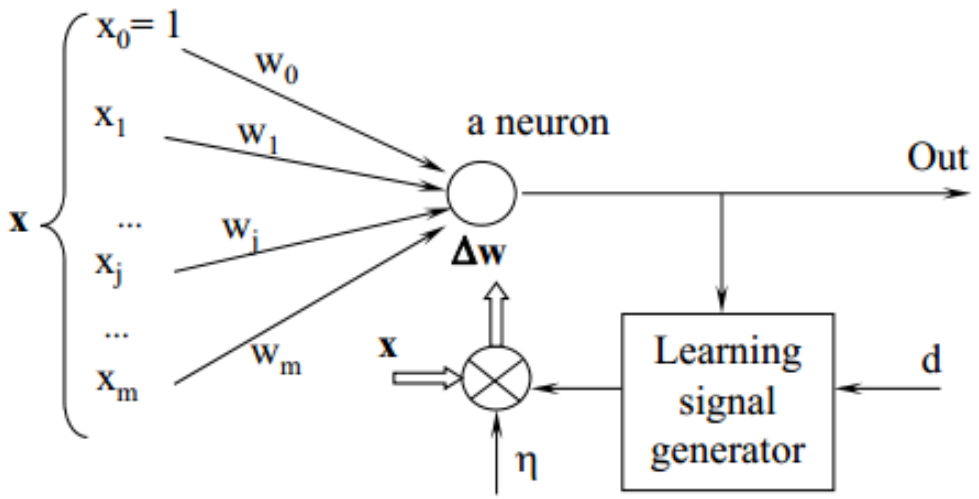
where η>0 is learning rate

- Learning signal r is a function of **w**, **x** and desired output value d

r = g(**w**,**x**,d)

- Generalized weight learning rules:

$$\Delta \boldsymbol{w}^{(t)} = \eta g(\boldsymbol{w}^{(t)}, \boldsymbol{x}^{(t)}, d^{(t)}) \boldsymbol{x}^{(t)}$$



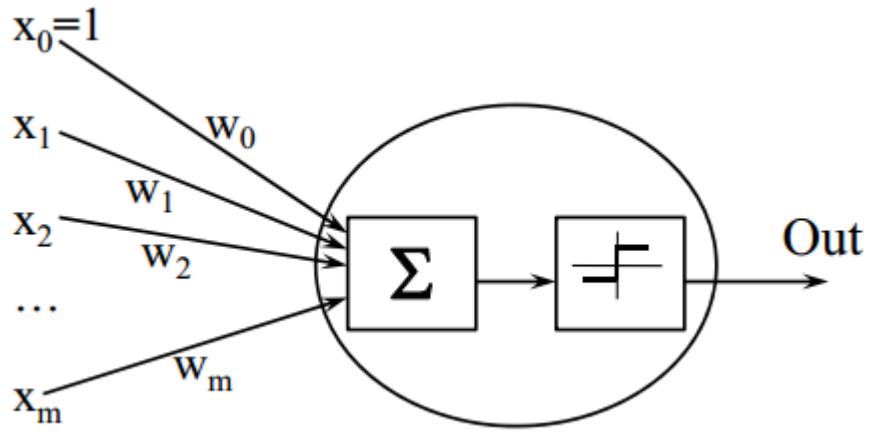
Note : x_j can be input signal or an output value from another neuron

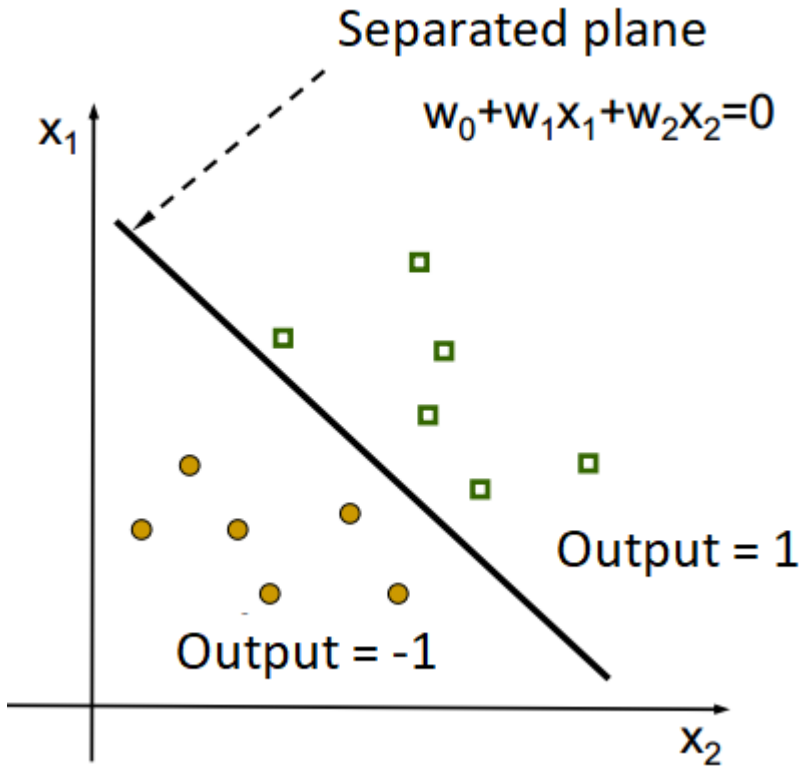
Perceptron

- A perceptron is the simplest of ANNs (include only 1 neuron)
- Use hard limiter activate function:

$$Out = sign\big(Net(w,x)\big) = sign\left(\sum_{j=0}^{m} w_j x_j\right)$$

- For an input x, output of a perceptron is:
  - 1 if Net(w,x)>0
  - -1 if other





Perceptron – Learning Algorithm

- With a training set D = {(**x**,d)}
  - **x** is an input vecto
  - d is a desired output (-1 or 1)
- The perceptron's learning process aims to determine a weight vector that allows the perceptron to produce the correct output value (-1 or 1) for each learning example.
- For an example x that is accurately classified by the perceptron, the weight vector w does not change
- If d=1 but the perceptron produces -1 (Out=-1), then w needs to be changed so that the value Net(w,x) increases
- If d=-1 but the perceptron produces 1 (Out=1), then w needs to be changed so that the value Net(w,x) decreases

Perceptron – Incremental Algorithm

Perceptron_incremental(D, η)

Initialize $\mathbf{w}$ ($w_i \leftarrow$ an initial (small) random value)

do

  for each training instance $(\mathbf{x}, d) \in D$

    Compute the real output value Out

    if (Out$\neq$d)

      $\mathbf{w} \leftarrow \mathbf{w} + \eta(d-Out)\mathbf{x}$

  end for

until all the training instances in D are correctly classified

return $\mathbf{w}$

Perceptron – Batch Algorithm

Perceptron_batch(D, η)

Initialize **w** ($w_i \leftarrow$ an initial (small) random value)

do

  $\Delta\mathbf{w} \leftarrow 0$

  for each training instance $(\mathbf{x}, d) \in D$

    Compute the real output value Out

    if (Out≠d)

      $\Delta\mathbf{w} \leftarrow \Delta\mathbf{w} + \eta(d-Out)\mathbf{x}$

  end for

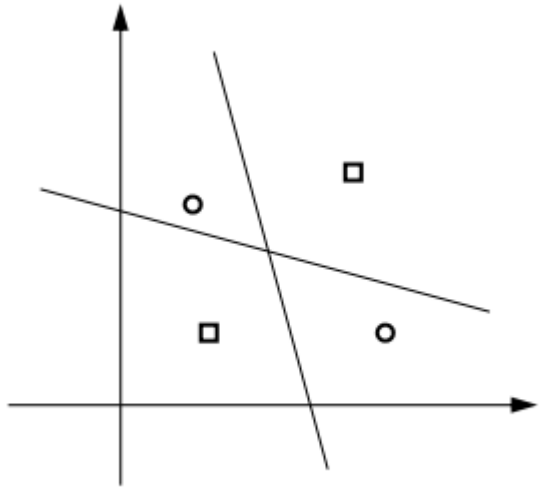
  $\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$

until all the training instances in D are correctly classified

return **w**

Perceptron – Limitation

- The learning algorithm for the perceptron is proven to converge if
  - The learning examples are linearly separable
  - Use a sufficiently small learning rate η
- The perceptron learning algorithm may not converge if the learning examples are not linearly separable
- Then, apply **Delta rules**
  - Ensure convergence to the most suitable approximation of the objective function
  - The delta rule uses a gradient descent strategy to find in the hypothesis space (weight vectors) a weight vector that best matches the learning examples



Error evaluation function

- Consider an ANN has n output neurons
- For an example (**x**,d), error value creates by current weight vecto **w** is

$$E_x(\boldsymbol{w}) = \frac{1}{2}\sum_{i=1}^{n}(d_i - Out_i)^2$$

- Error value ceates by current weight vecto **w** for all training set D is:

$$E_D(\boldsymbol{w}) = \frac{1}{|D|}\sum_{i=1}^{n} E_x(\boldsymbol{w})$$

Gradient descent

- Gradient of E (∇E) is a vecto:
  - Has an upward direction (slope)
  - Has a length proportional to the slope
- The gradient ∇E determines the direction that causes the steepest increase in the error value E

$$\nabla E(\boldsymbol{w}) = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \dots, \frac{\partial E}{\partial w_N}\right)$$

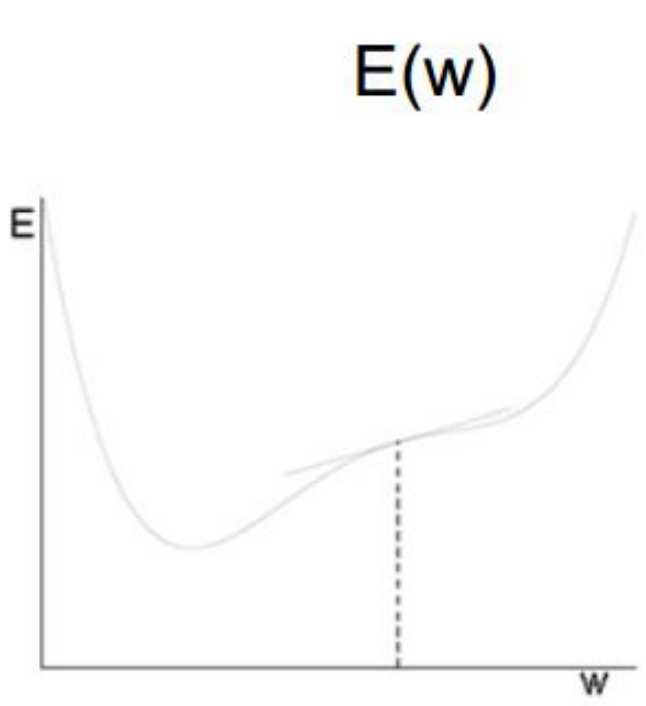
where N is number of weight w in the network

- So that, the direction causing the steepest decrease is the negative value of the gradient of E

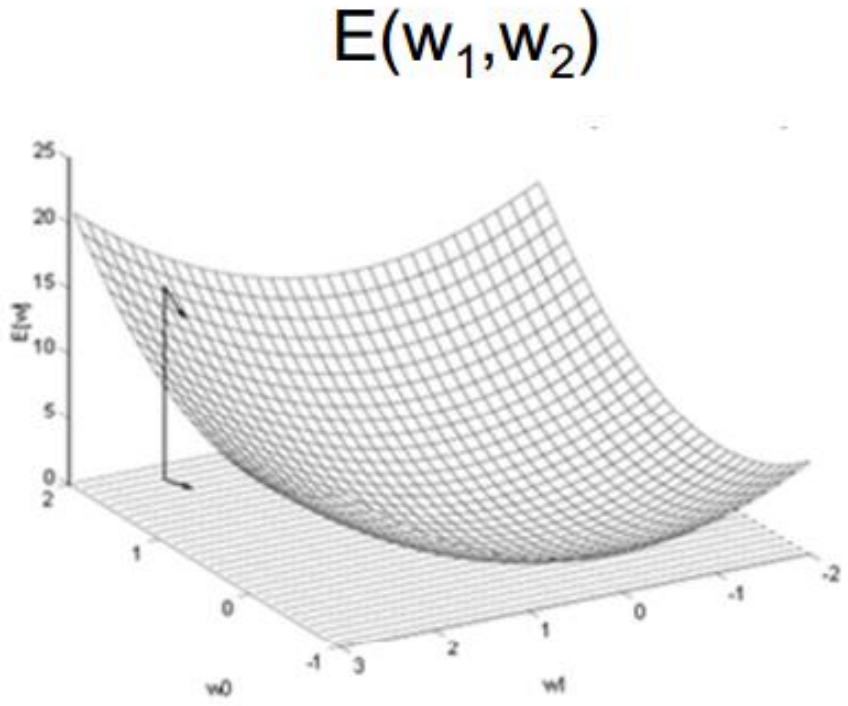
$$\nabla \boldsymbol{w} = -\eta . \nabla E(\boldsymbol{w}); \qquad \nabla w_i = -\eta . \frac{\partial E}{\partial w_i} \quad , \forall i = 1, \dots, N$$

=>Requirement: The activate functions used in the network must be continuous functions with respect to the weights and have continuous derivatives.

Gradient descent - visualization



One direction space



Two direction space

Gradient_descent_incremental (D, η)

Initialize $\mathbf{w}$ ($w_i \leftarrow$ an initial (small) random value)

do

  for each training instance $(\mathbf{x}, d) \in D$

    Compute the network output

    for each weight component w_i

      $w_i \leftarrow w_i - \eta (\partial E_{\mathbf{x}} / \partial w_i)$

    end for

  end for

until (stopping criterion satisfied)

return $\mathbf{w}$

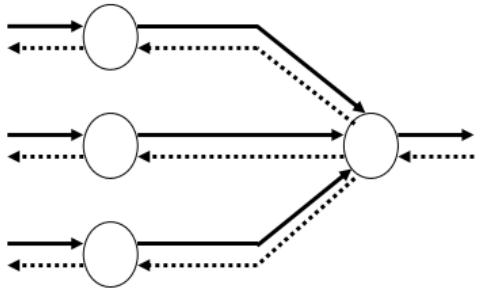
Stopping criterion: number of study cycles (epochs), error threshold…

Multi-layer ANN and back propagation algorithm

- A perceptron can only represent a linear separable function
- A multi-layer neural network (NN) learned by the back-propagation (BP- algorithm) can represent a highly non-linear separation function
- The BP learning algorithm is used to learn the weights of a multilayer neural network
  - Fixed network structure (neurons and the links between them are fixed)
  - For each neuron, the action function must have a continuous derivative
- The BP algorithm applies the gradient descent strategy in the weights update rule to minimize error

.

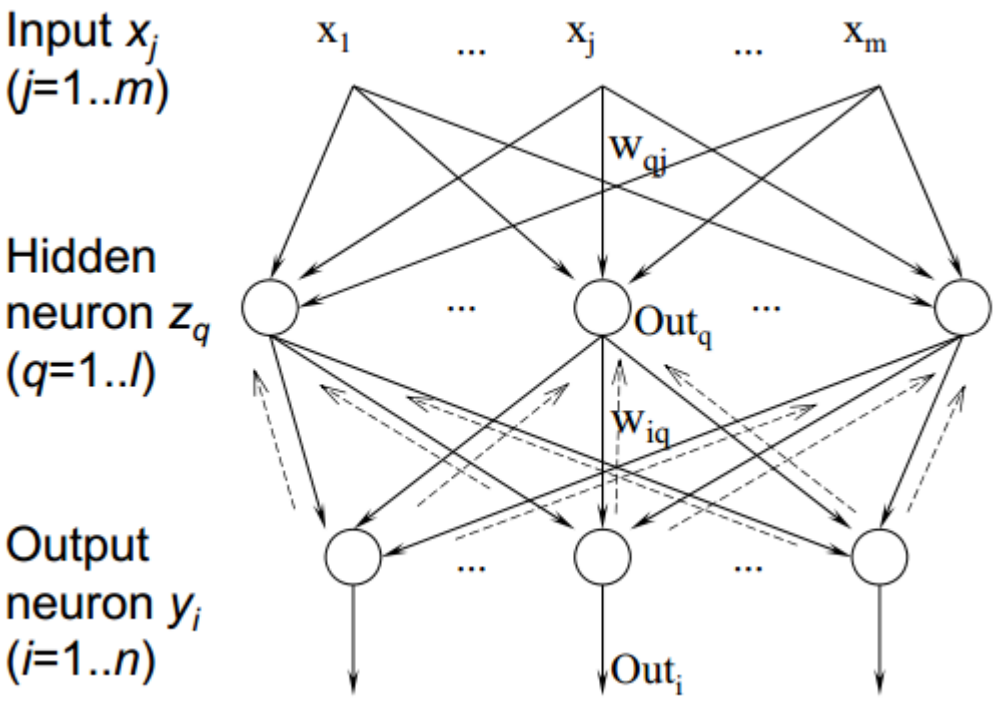
Back propagation algorithm



- The backpropagation learning algorithm searches for a vector of weights that minimizes the overall error of the system for the learning set
- BP includes 2 steps:
  1. Signal forward step:
     - The input signals are forward propagated from the input layer to the output layer (passing through hidden layers).
  2. Error backward step:
     - Based on the desired output value of the input vector, the system calculates the error value
     - From the output layer, the error value is propagated back through the network, from layer to layer, until the input layer
     - Error back-propagation is performed through calculating (recursively) the local gradient value of each neuron

BP-algorithm network structure

- Consider 3-layer NN:
  - m input signal x_j
  - l hidden layer neural z_q
  - n output neurons y_i
- w_{qj} is the weight of the link from the input signal x_j to the hidden layer neuron z_q
- w_{iq} is the weight of the link from the hidden layer neuron z_q to output neuron y_i
- Out_q is the (local) output value of the hidden layer neuron z_q
- Out_i is the output value of the network corresponding to the output neuron y_i



BP-algorithm: forward propagation (1)

- For each example x:
  - The input vector x is propagated from the input layer to the output layer
  - The network will produce an actual output value *Out* (which is a vector of Out_i values)
- For each input vecto x:
  - a neuron z_q in the hidden layer will receive a net input value of:

    $Net_q = \sum_{j=1}^{m} w_{qj}\, x_j$
  - and create a local output of:

    $Out_q = f\left(Net_q\right) = f\left(\sum_{j=1}^{m} w_{qj}\, x_j\right)$

  where f is an activate function

BP-algorithm: forward propagation (2)

- The net input value of neuron y_i at the output layer:

$$Net_i = \sum_{q=1}^{l} w_{iq}\, Out_q = \sum_{q=1}^{l} w_{iq}\, f\left(\sum_{j=1}^{m} w_{qj}\, x_j\right)$$

- Neuron y_i create an output value Out_i :

$$Out_i = f(Net_i) = f\left(\sum_{q=1}^{l} w_{iq}\, f\left(\sum_{j=1}^{m} w_{qj}\, x_j\right)\right)$$

- Vecto created of Out_i, $i=1\ldots n$, is the output of the network for vecto **x**

BP-algorithm: backward propagate (1)

- For each example **x**
  - Error signals due to the difference between the desired output value d and the actual output value Out are calculated
  - These error signals are back-propagated from the output layer to the front layers, to update the weights.
- To consider error signals and their back propagation, it is necessary to define an error evaluation function

$$E(w) = \frac{1}{2}\sum_{i=1}^{n}(d_i - Out_i)^2 = \frac{1}{2}\sum_{i=1}^{n}(d_i - f(Net_i))^2$$

$$= \frac{1}{2}\sum_{i=1}^{n}(d_i - f(\sum_{q=1}^{l} w_{iq}\, Out_q))^2$$

BP-algorithm: backward propagate (2)

- Apply Gradient-descent, weights from hidden to output layer are update by:

$$\nabla w_{iq} = -\eta . \frac{\partial E}{\partial w_{iq}}$$

- Apply the chain derivative rule we have

$$\nabla w_{iq} = \eta \delta_i Out_q$$

- δ_i is the error signal of neuron y_i at the output layer

$$\delta_i = [d_i - Out_i][f'(Net_i)]$$

BP-algorithm: backward propagate (3)

- To update the weights of the links from the input layer to the hidden layer, we also apply the gradient-descent method and the derivative chain rule

$$\nabla w_{qj} = -\eta . \frac{\partial E}{\partial w_{qj}} = \eta \delta_q x_j$$

- δ_q is the error signal of neuron z_q at the hidden layer

$$\delta_q = f'(Net_q) \sum_{i=1}^{n} \delta_i w_{iq}$$

- According to the above formulas for calculating error signals δ_i and δ_q, the error signal of a neuron in the hidden layer is different from the error signal of a neuron in the output layer.
- Due to this difference, the weight update procedure in the BP algorithm is called the ***generalized delta learning rule***

BP-algorithm: backward propagate (4)

- The error signal δ_q of neuron z_q in the hidden layer is determined by:
  - The error signals δ_i of neurons y_i in the output layer (to which neuron z_q is linked) and
  - The coefficients are w_{iq} weights
- Important features of the BP algorithm: **The weight update rule is local**
  - To update the weight of a link, the system only needs to use the values at the two ends of that link
- The general form of the weight updating rule in the BP algorithm is

$$\Delta w_{ab} = \eta \delta_a x_b$$

Back_propagation_incremental Alg (1)

Back_propagation_incremental(D, η)

Neural network include Q layers, q=1,2,…,Q

Net_i^q and Out_i^q are net input and output of neural *i* at layer q

Network has *m* input signals and *n* output neural

w_{ij}^q is the weight of the link from neural j of the layer (q-1) to neural q of the layer q

Step 0 (Initalization)

Choose error threshold E_{th}

Initialize the initial values of the weights with small random values

Let E=0

Step 1 (Start a learning period)

Apply the input vector of the learning example k to the input layer (q=1)

$Out_i^q = Out_i^1 = x_i^{(k)}, \quad \forall i$

Step 2 (forward propagation)

Forward propagation of input signals through the network, until the network output values are received (at the output layer) Out_i^q

Back_propagation_incremental Alg (2)

Step 3 (Output Error Coutting)

Calculate the output error of the network and the error signal of each neuron δ_i^q in the output layer

$$E = E + \frac{1}{2}\sum_{i=1}^{n}(d_i^{(k)} - Out_i^q)$$

$$\delta_i^q = \left(d_i^{(k)} - Out_i^q\right) f'(Net_i^q)$$

Step 4 (Error back-propagation)

Error back-propagation to update the weights and calculate the error signals δ_i^{q-1} for the above layers

$$\Delta w_{ij}^q = \eta \delta_i^q Out_j^{q-1}; \qquad w_{ij}^q = w_{ij}^q + \Delta w_{ij}^q$$

$$\delta_i^{q-1} = f'(Net_i^{q-1}) w_{ji}^q \delta_j^q$$

Step 5 (Check the end of a learning cycle – epoch)

Check if the entire learning set has been used (a learning cycle has been completed – epoch)

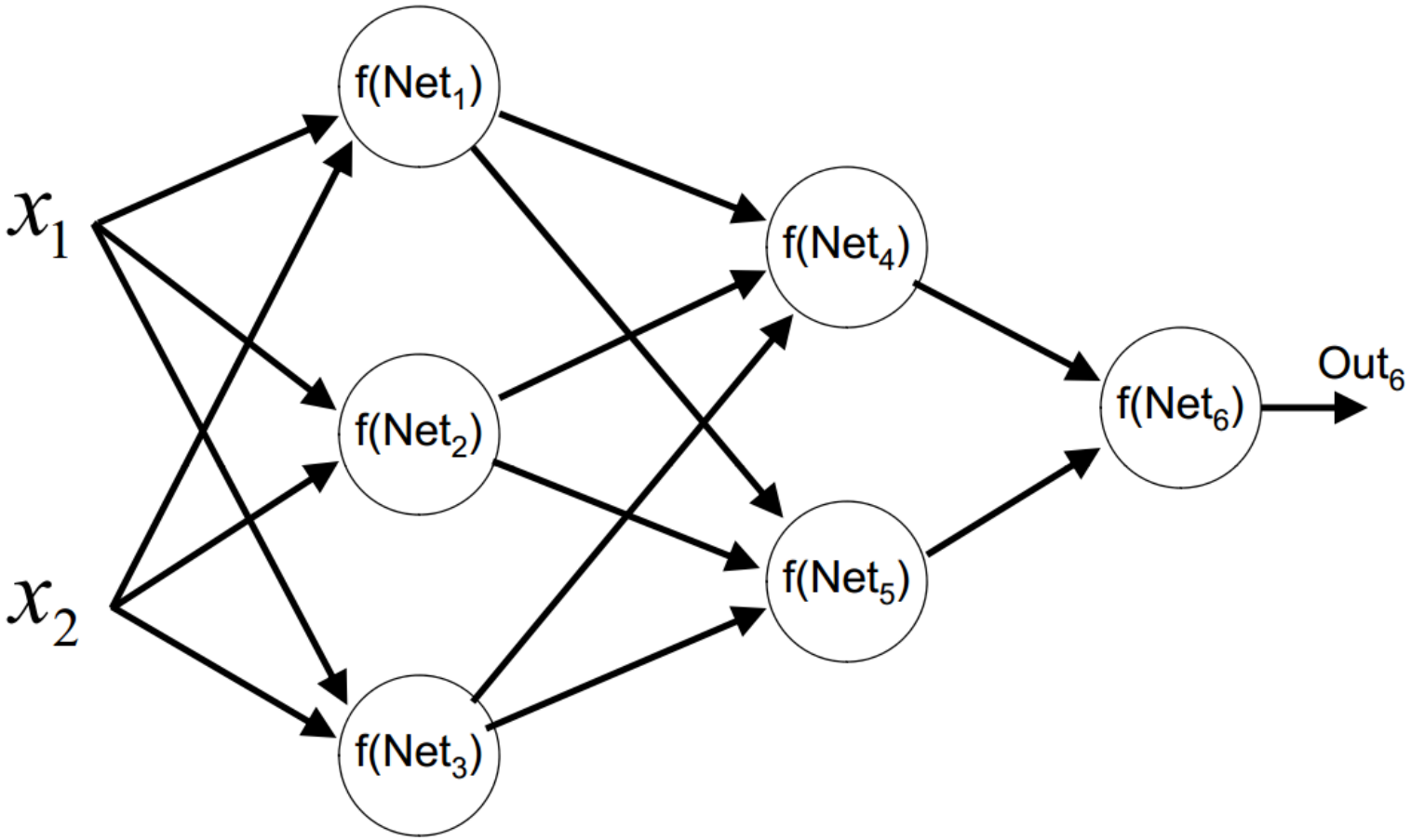
If the entire study set has been used, go to Step 6; otherwise, go to Step 1

Step 6 (Check for overall errors)

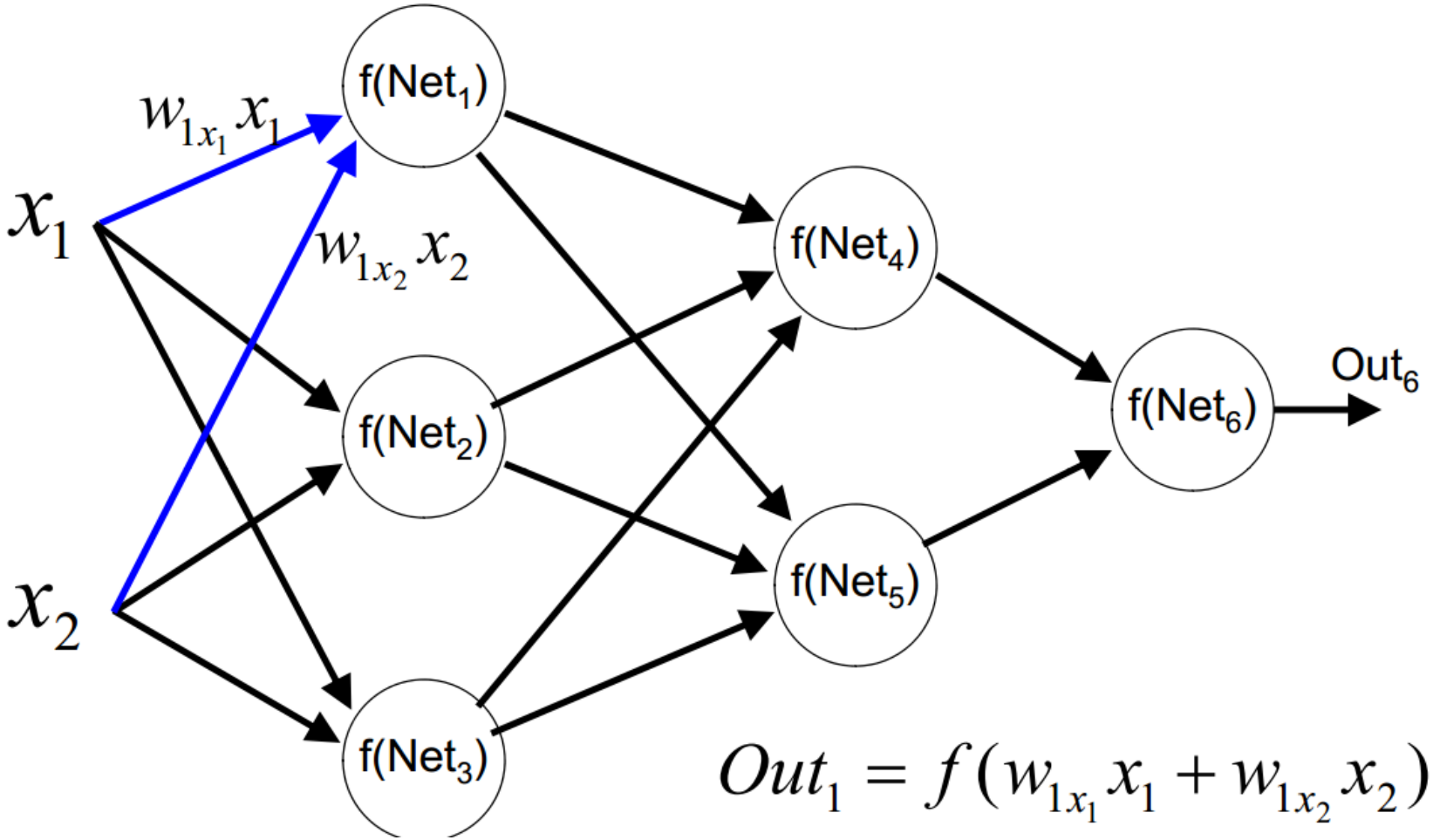
If the overall error E is less than the acceptable error threshold ($<E_{th}$), the learning process ends and the learned weights are returned;

Otherwise, reassign E=0, and start a new learning cycle (return to Step 1).

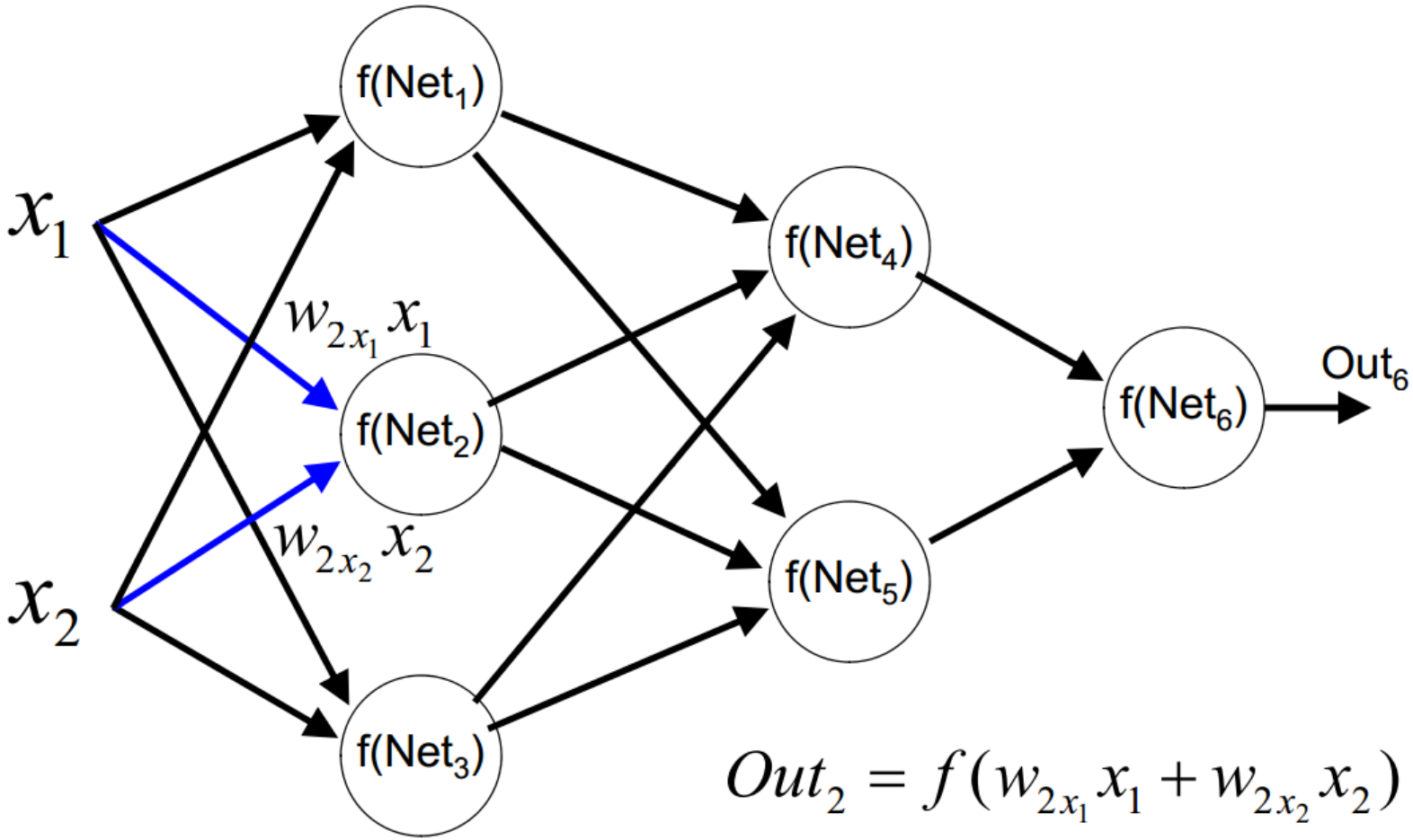
Forward propagation visualization (1)



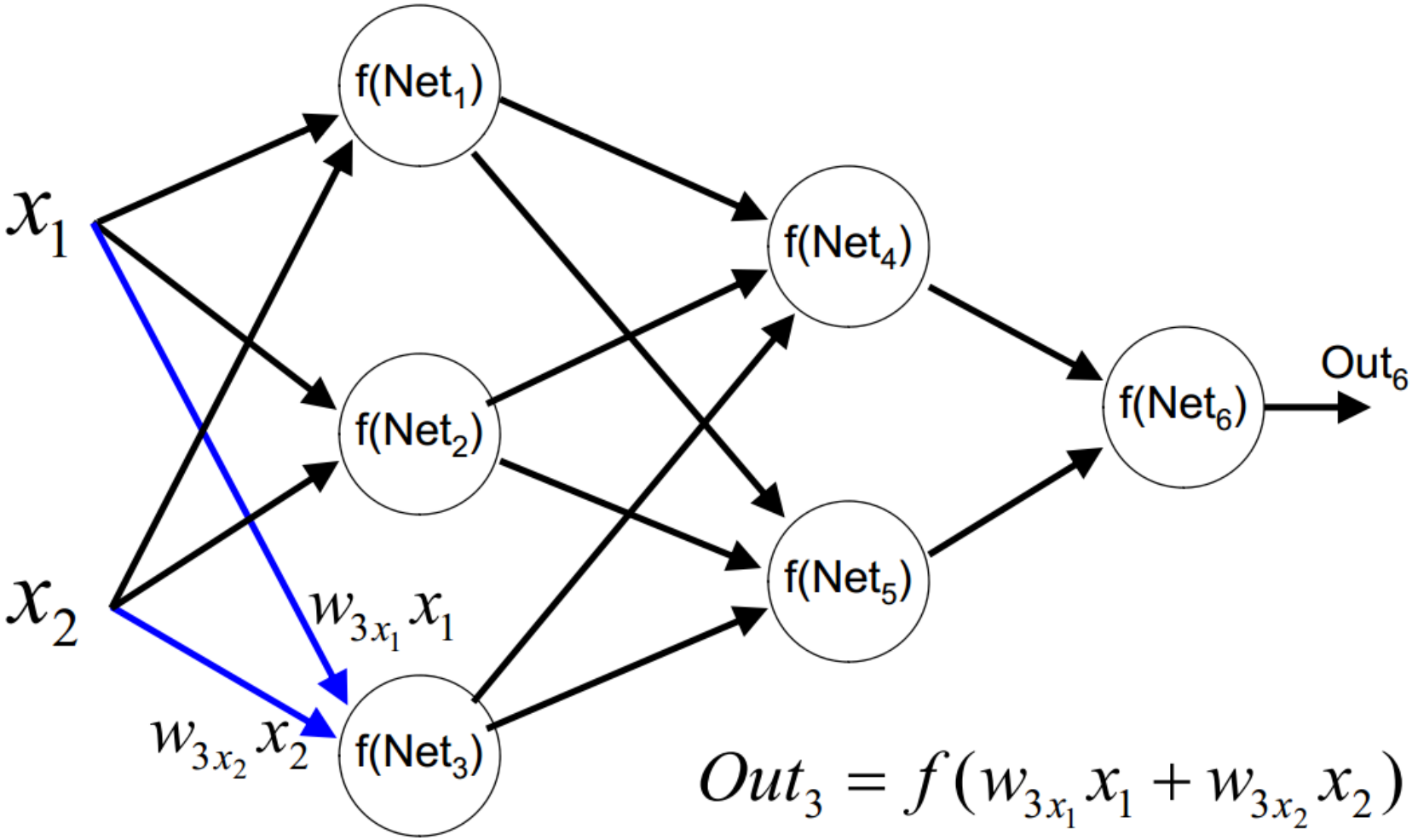
Forward propagation visualization (2)



Forward propagation visualization (3)

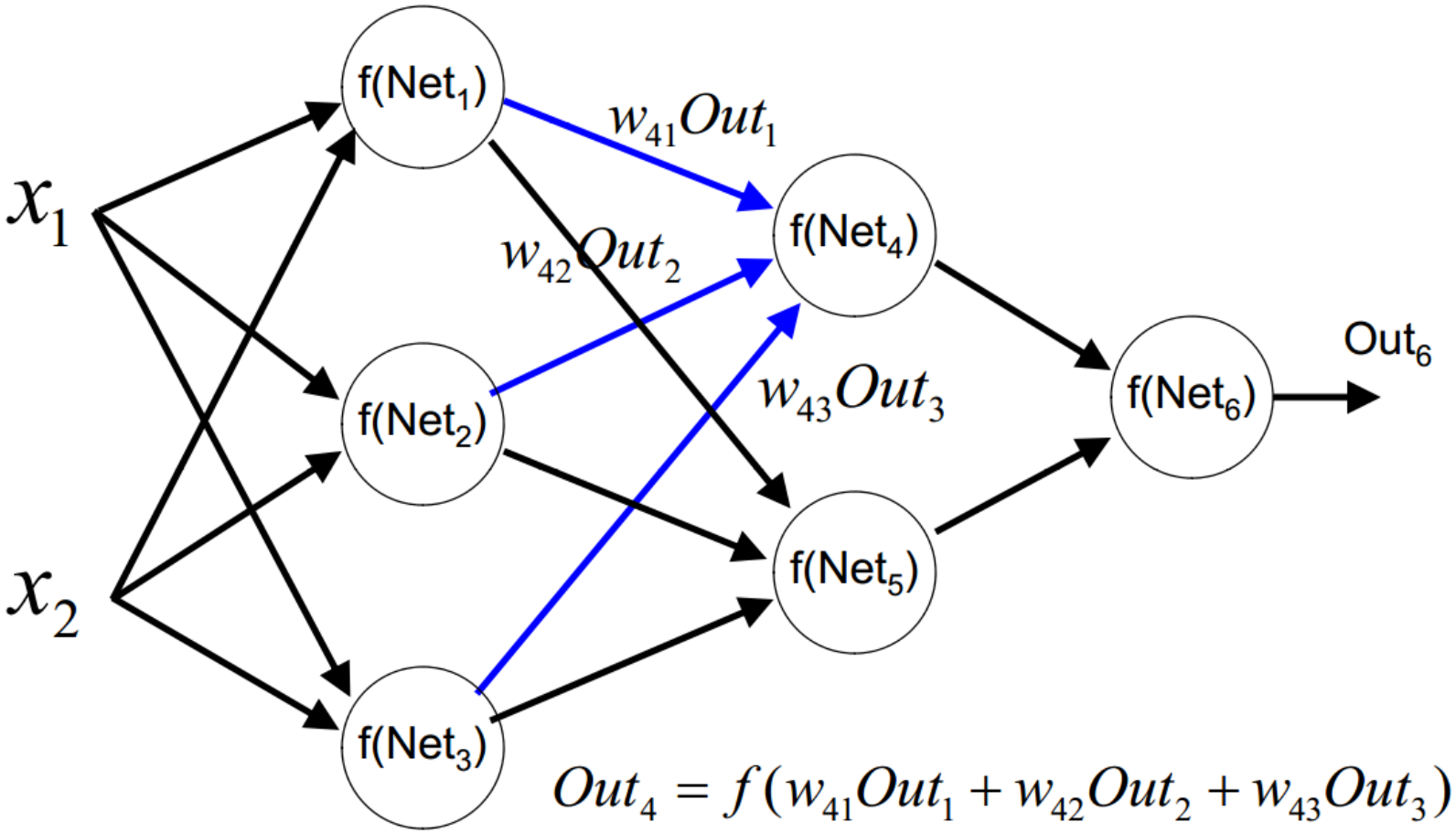


Forward propagation visualization (4)



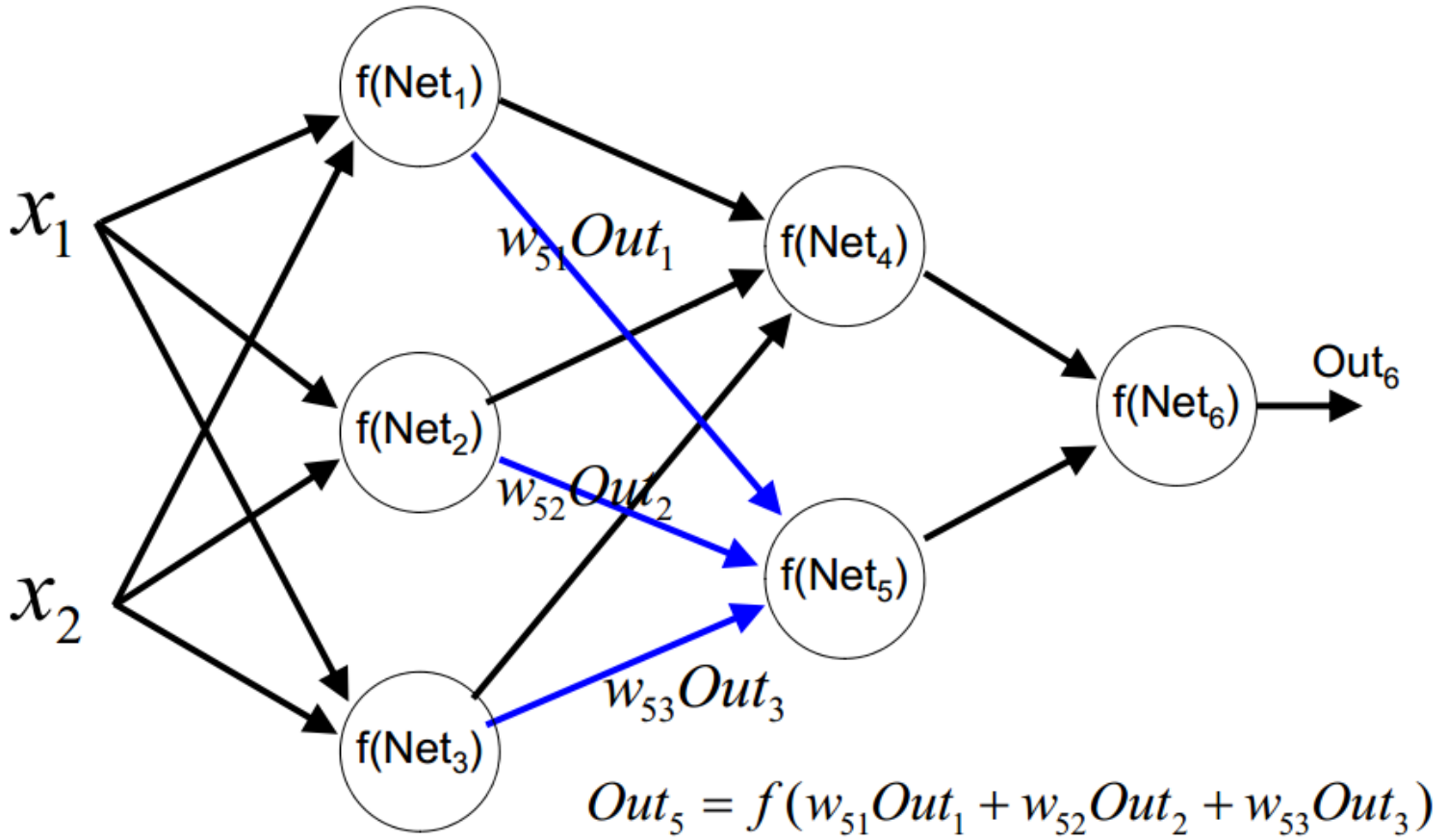
$$Out_3 = f(w_{3x_1} x_1 + w_{3x_2} x_2)$$

Forward propagation visualization (5)



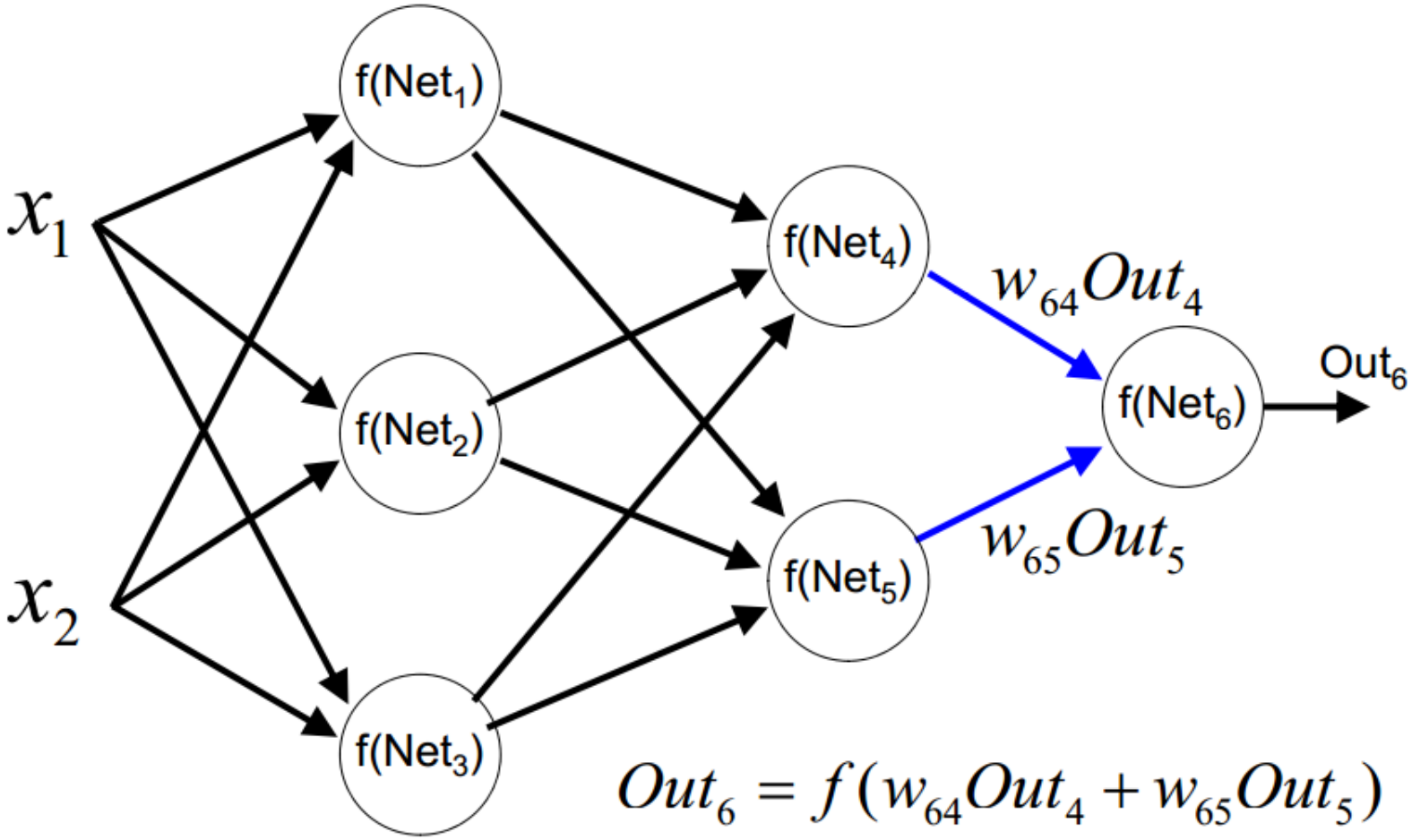
$$Out_4 = f(w_{41}Out_1 + w_{42}Out_2 + w_{43}Out_3)$$

Forward propagation visualization (6)

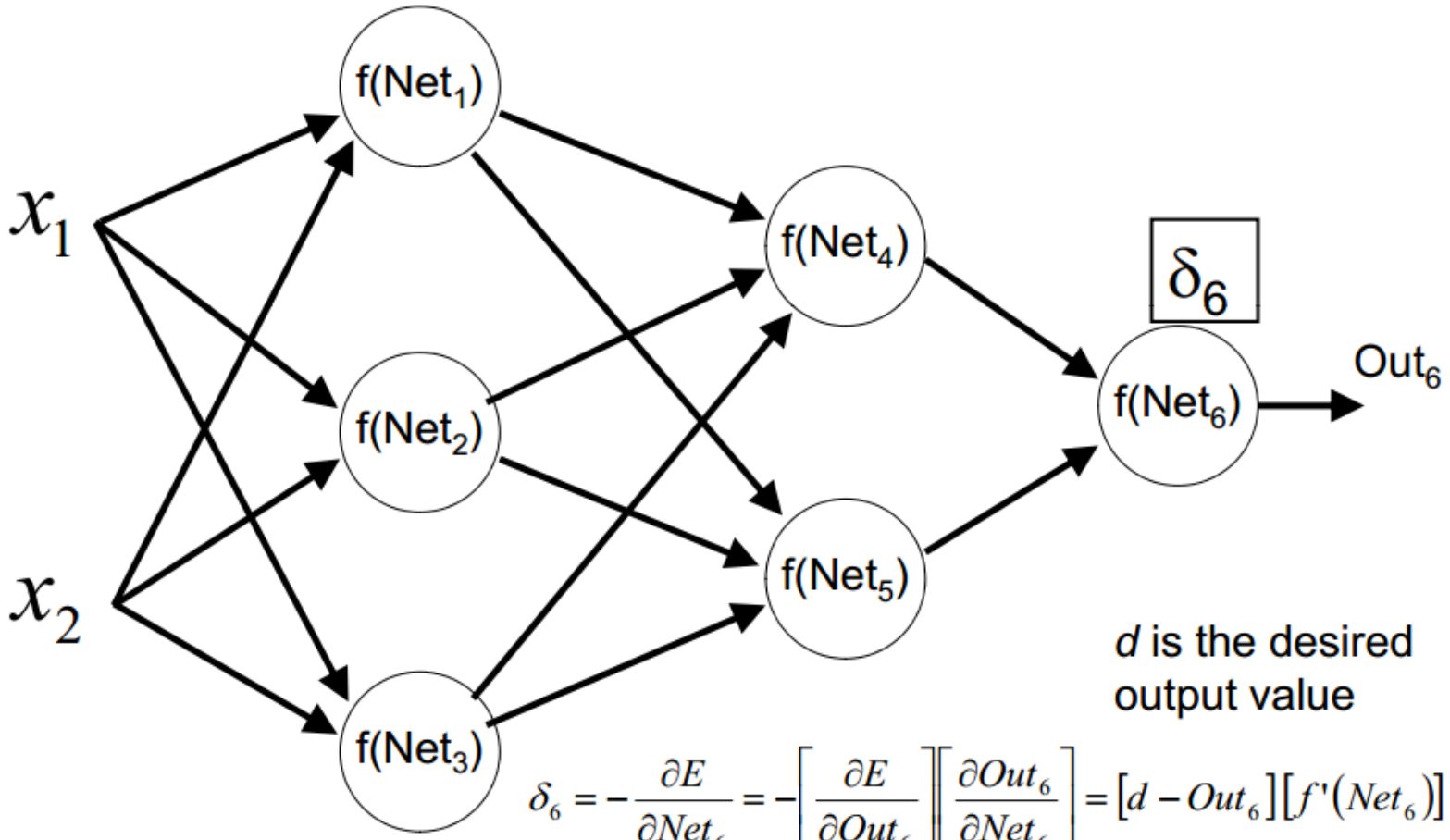


$$Out_5 = f(w_{51}Out_1 + w_{52}Out_2 + w_{53}Out_3)$$

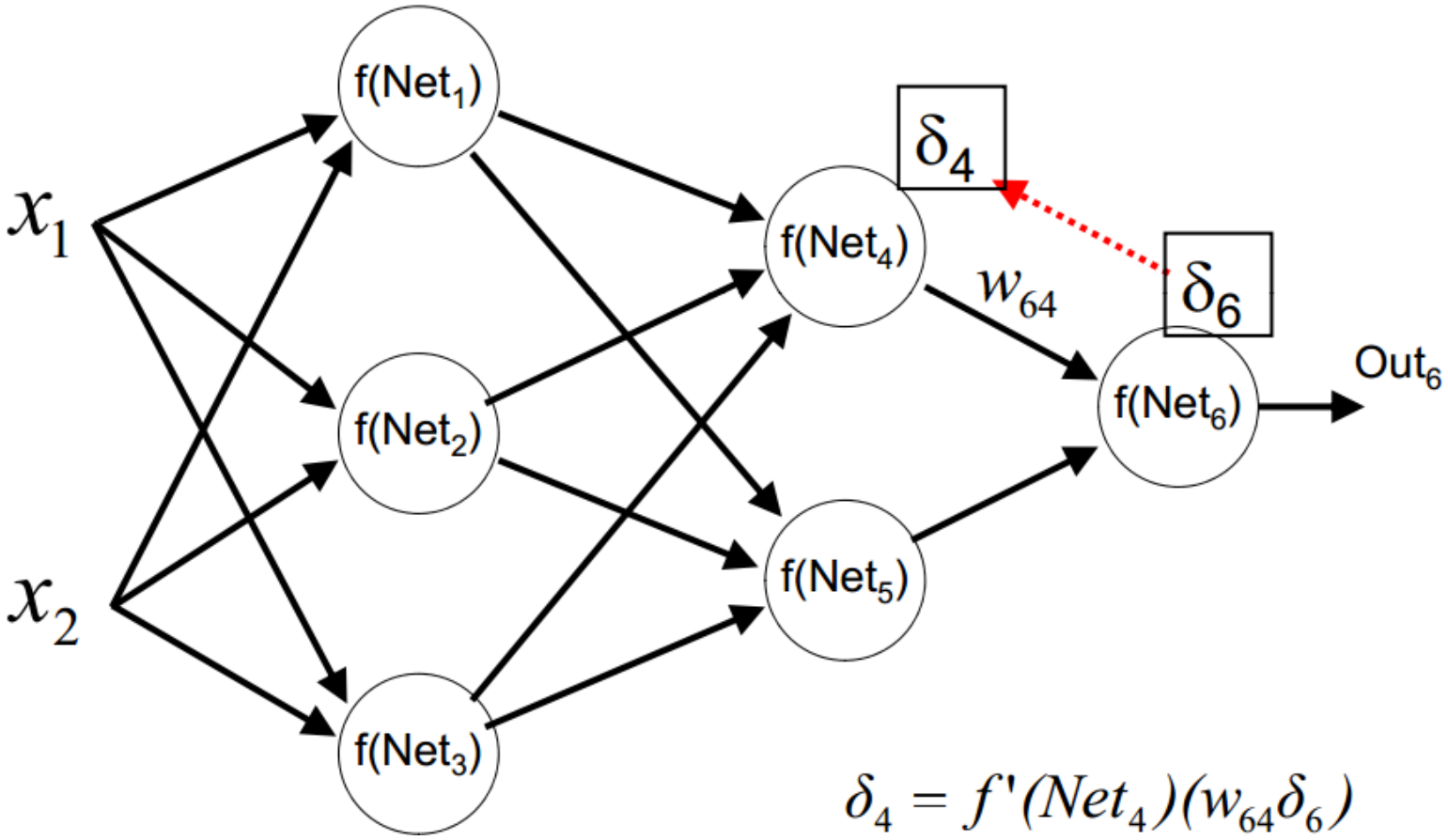
Forward propagation visualization (7)



Error Counting

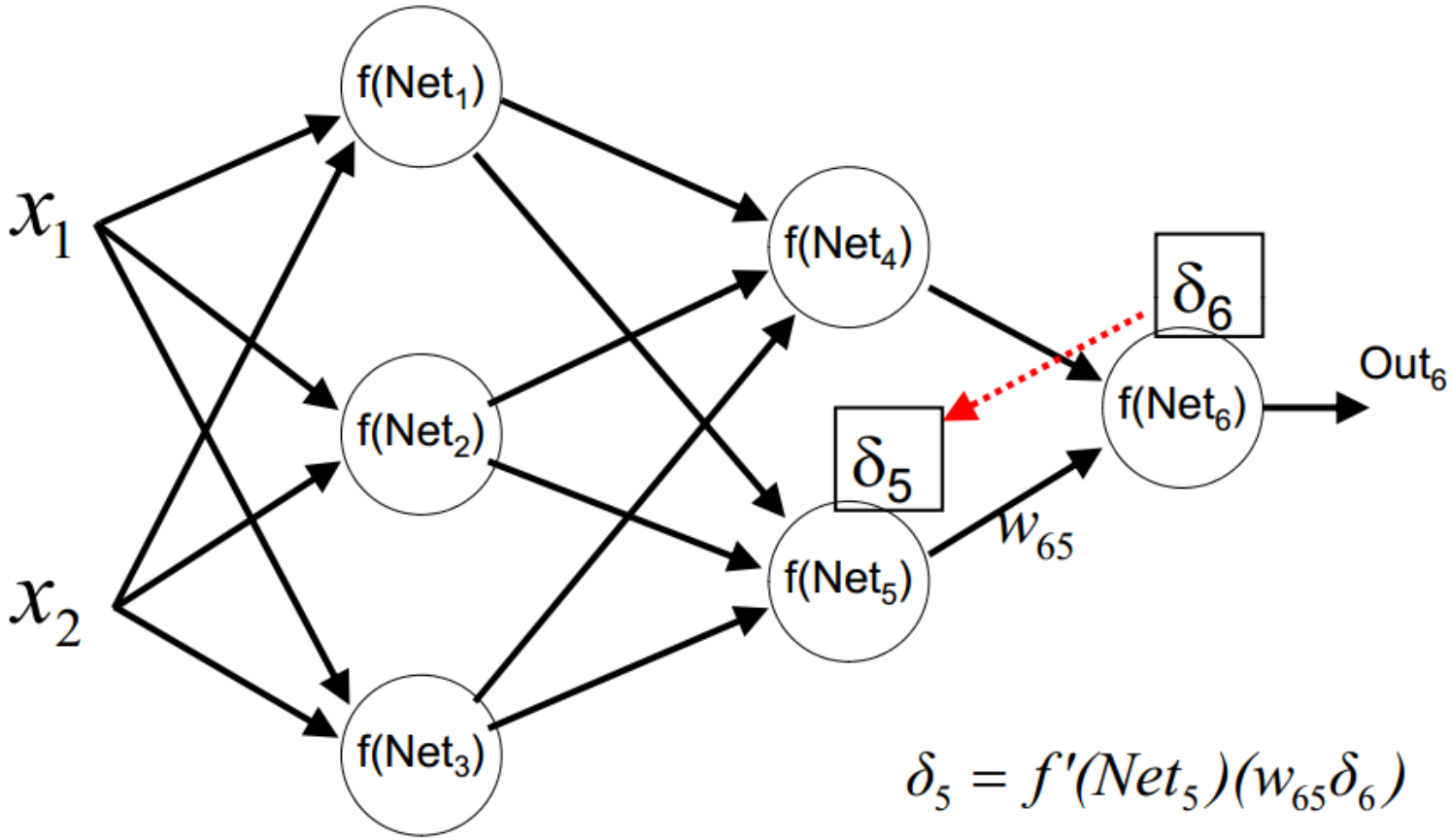


Back propagation (1)



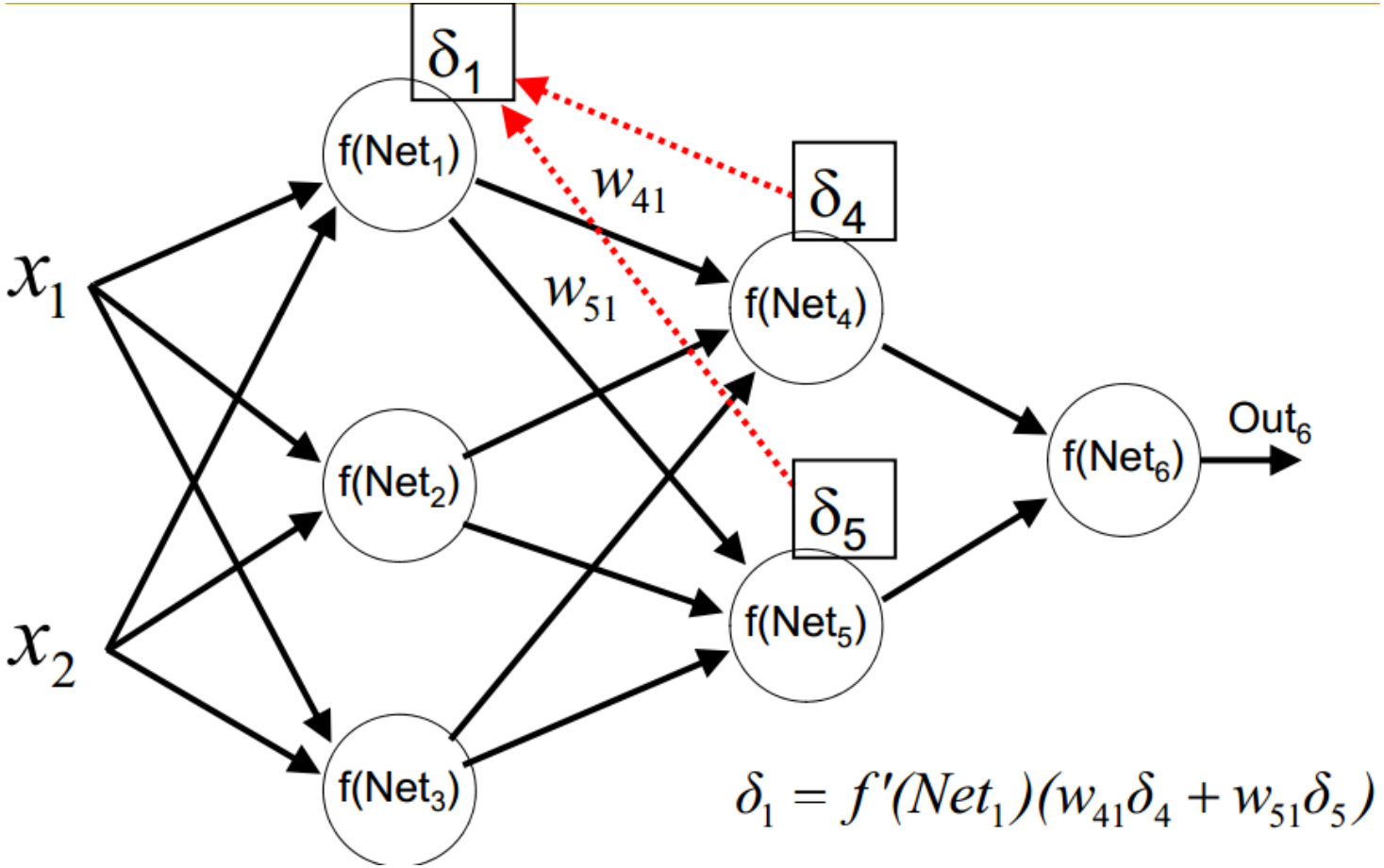
$$\delta_4 = f'(Net_4)(w_{64}\delta_6)$$

Back propagation (2)



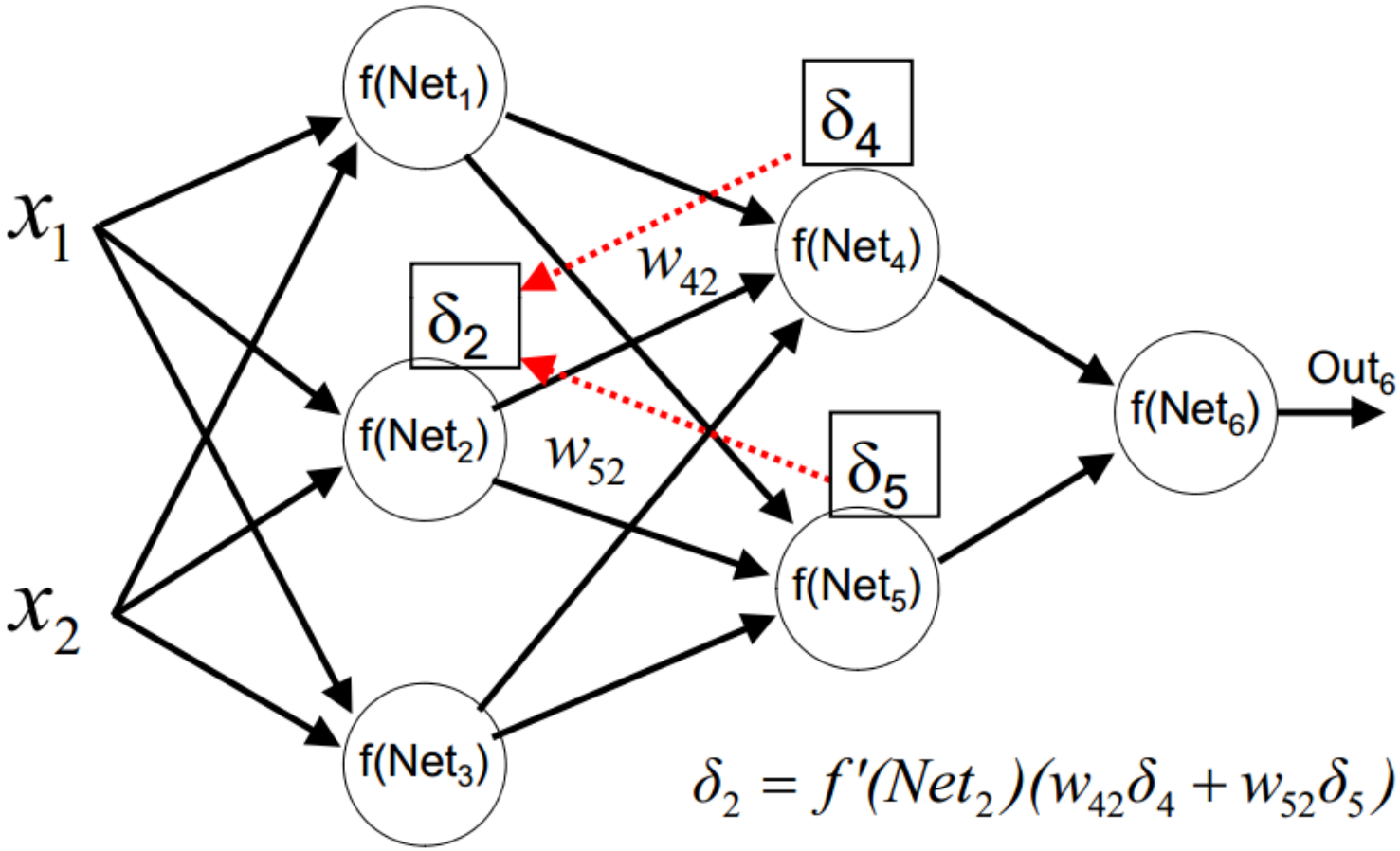
$$\delta_5 = f'(Net_5)(w_{65}\delta_6)$$

Back propagation (3)

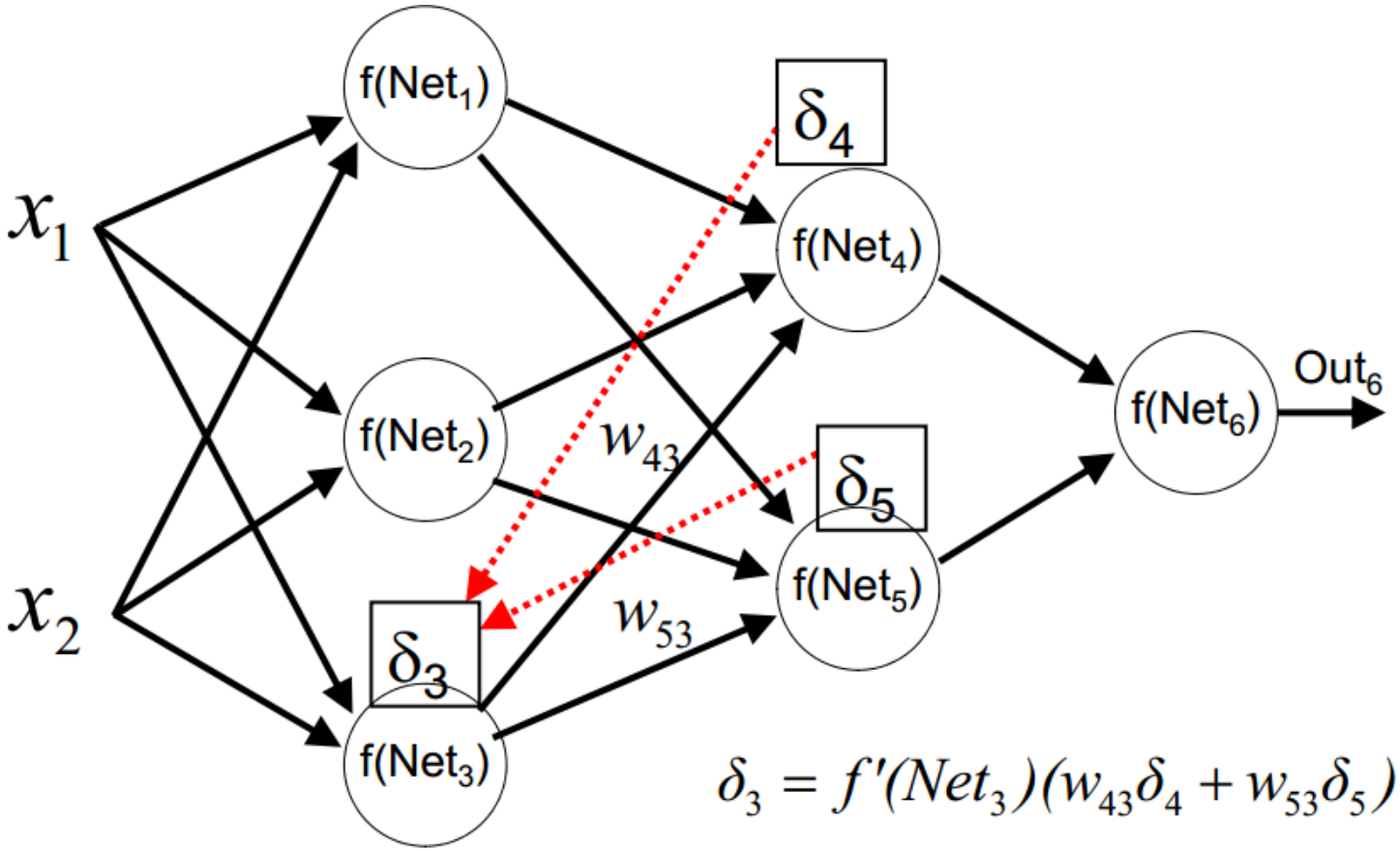


$$\delta_1 = f'(Net_1)(w_{41}\delta_4 + w_{51}\delta_5)$$

Back propagation (4)

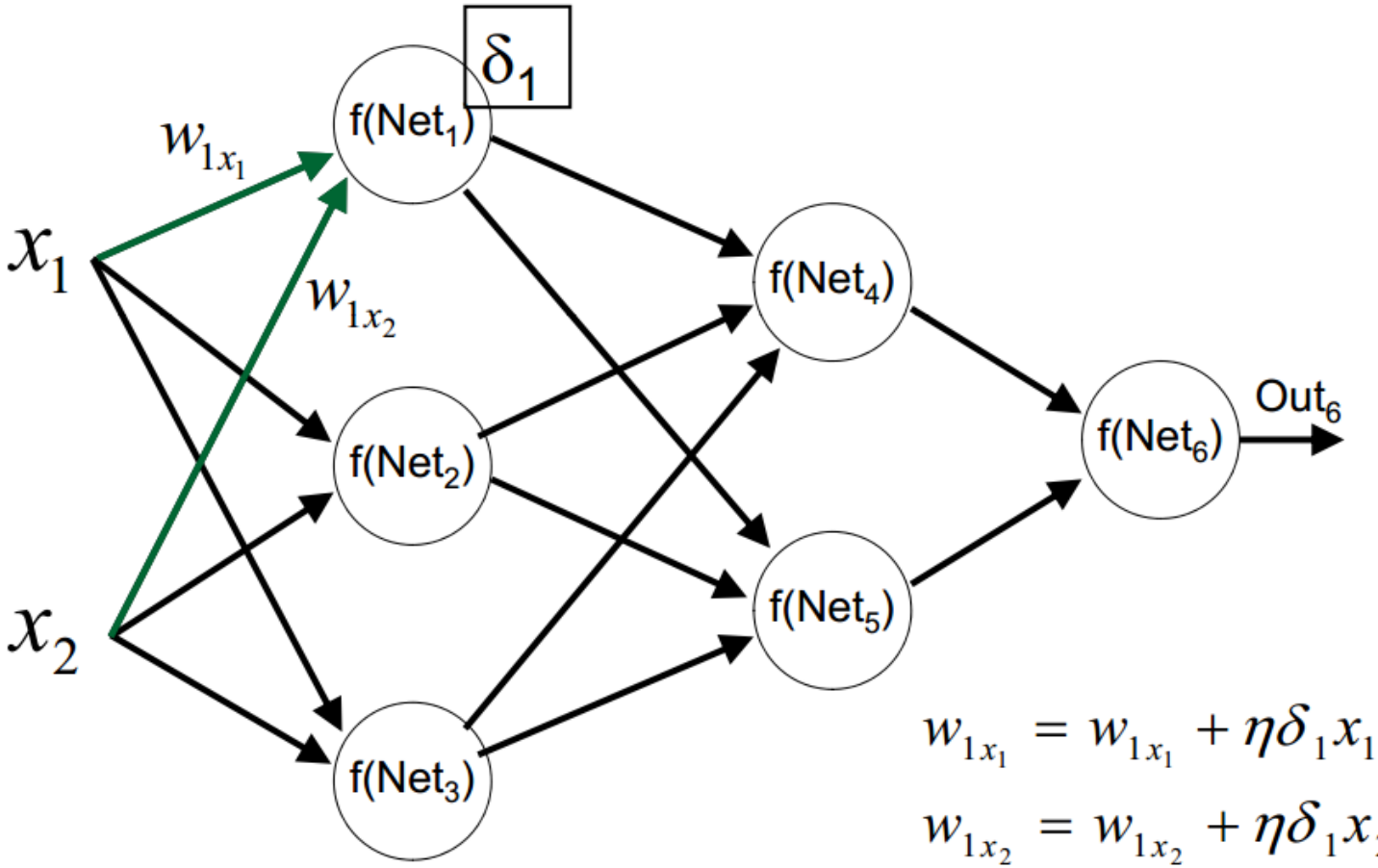


Back propagation (5)



$$\delta_3 = f'(Net_3)(w_{43}\delta_4 + w_{53}\delta_5)$$

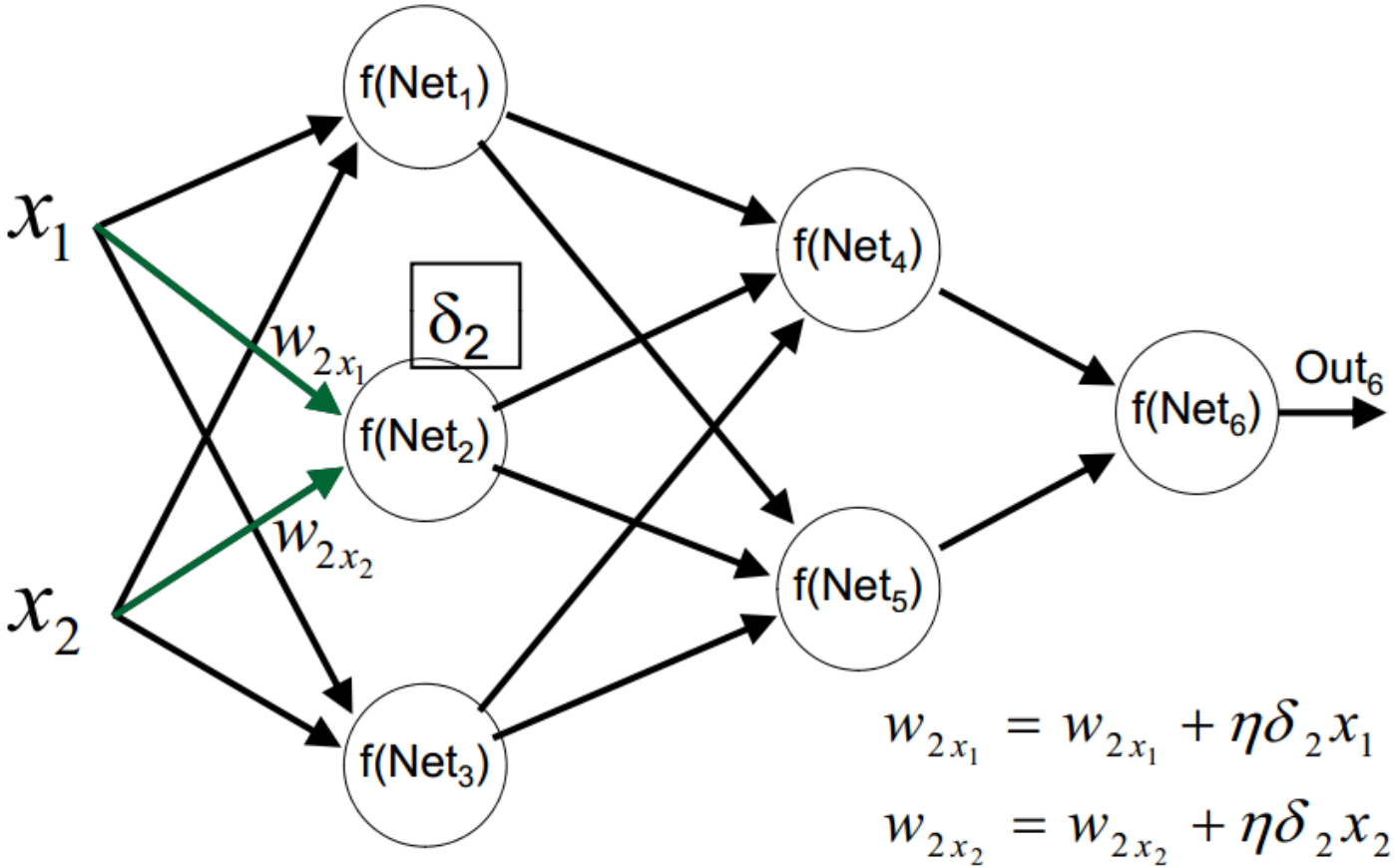
Weights updating (1)



$$w_{1x_1} = w_{1x_1} + \eta\delta_1 x_1$$

$$w_{1x_2} = w_{1x_2} + \eta\delta_1 x_2$$

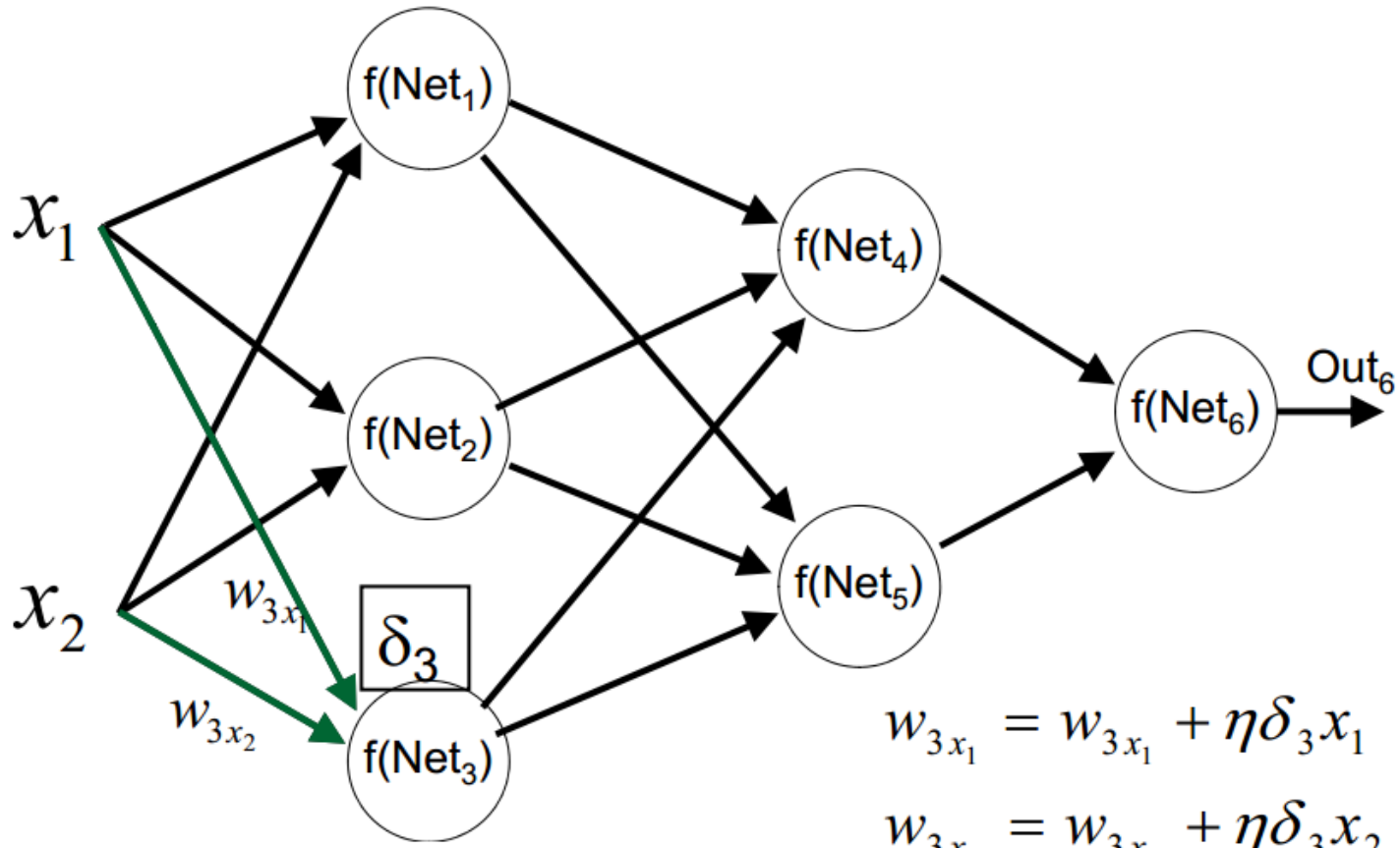
Weights updating (2)



$$w_{2x_1} = w_{2x_1} + \eta\delta_2 x_1$$

$$w_{2x_2} = w_{2x_2} + \eta\delta_2 x_2$$

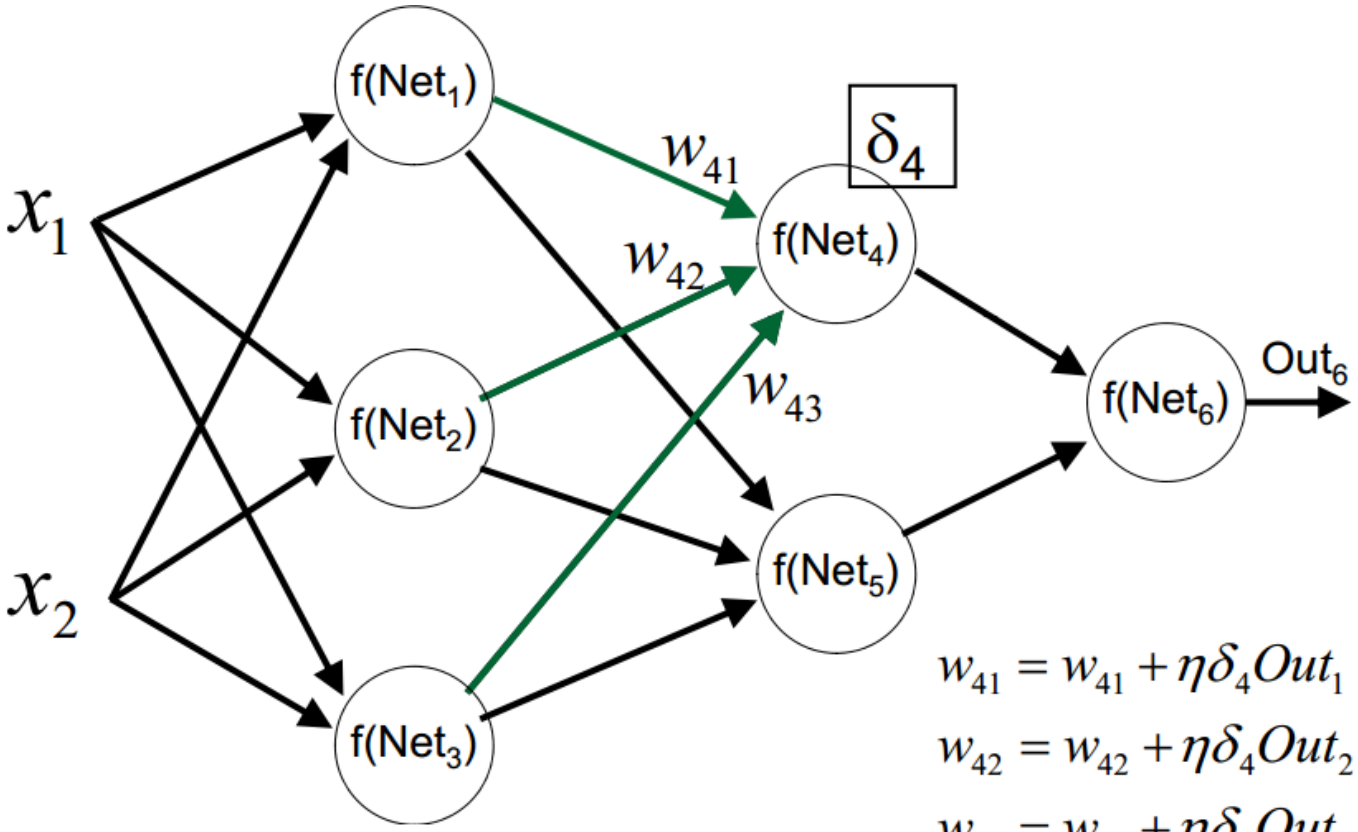
Weights updating (3)



$$w_{3x_1} = w_{3x_1} + \eta\delta_3 x_1$$

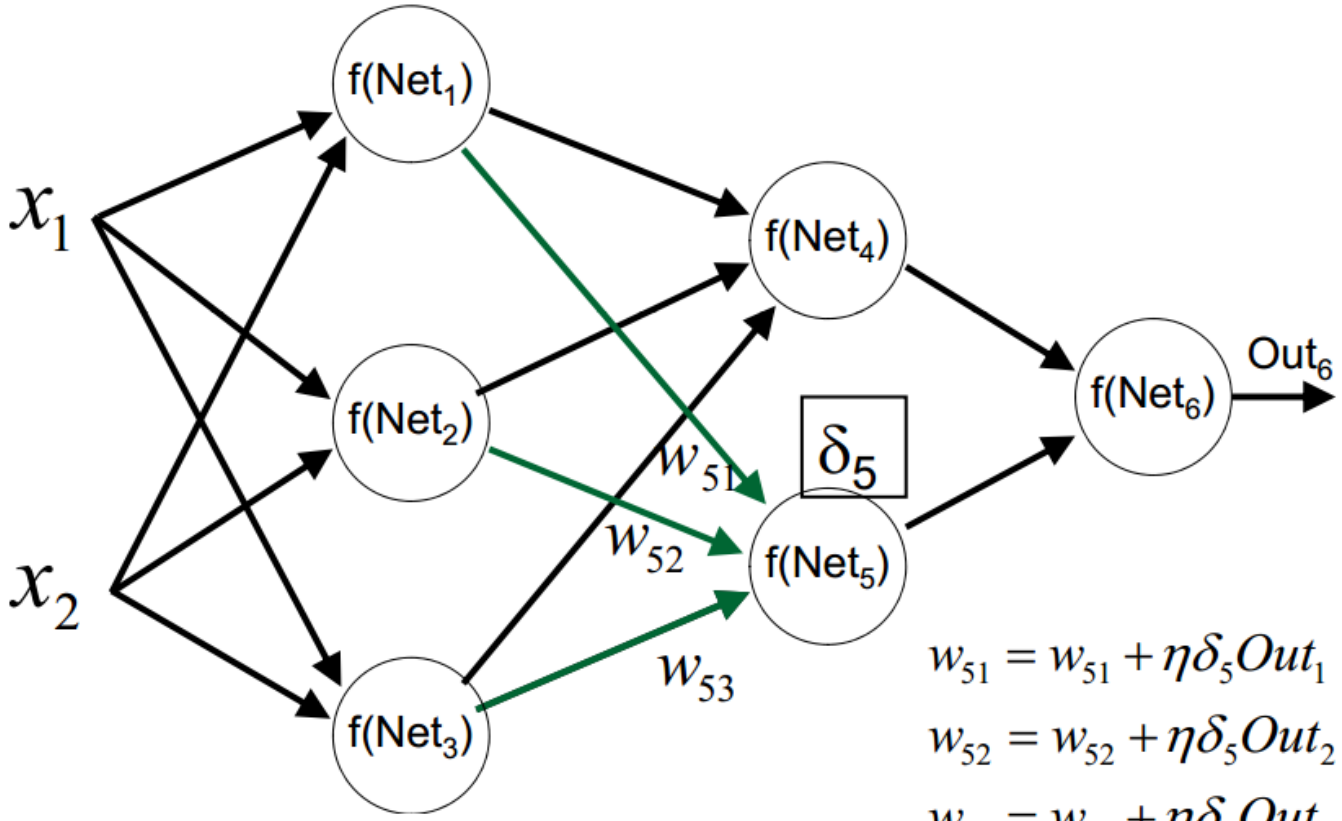
$$w_{3x_2} = w_{3x_2} + \eta\delta_3 x_2$$

Weights updating (4)



$$w_{41} = w_{41} + \eta\delta_4 Out_1$$
$$w_{42} = w_{42} + \eta\delta_4 Out_2$$
$$w_{43} = w_{43} + \eta\delta_4 Out_3$$

Weights updating (5)

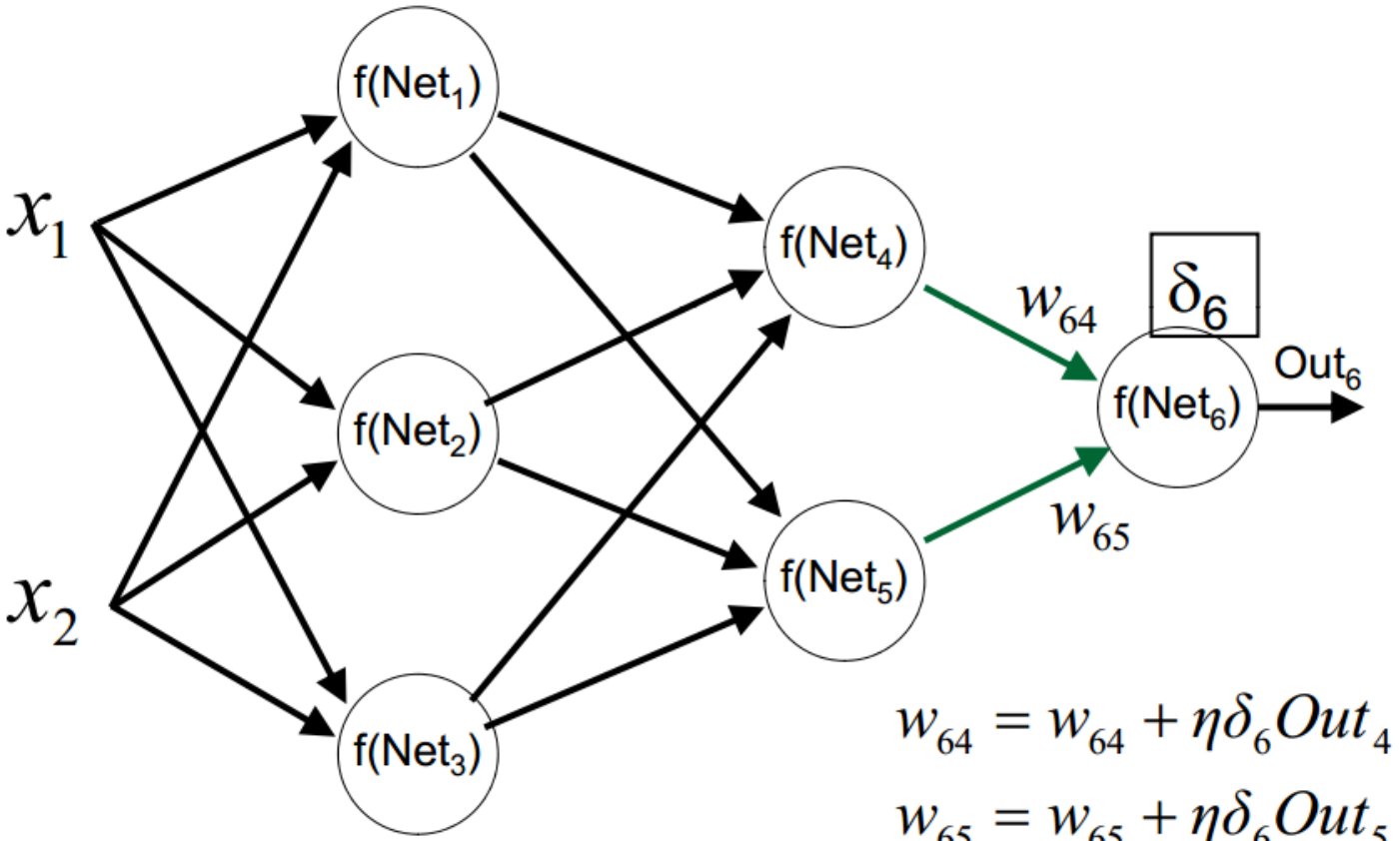


$$w_{51} = w_{51} + \eta\delta_5 Out_1$$

$$w_{52} = w_{52} + \eta\delta_5 Out_2$$

$$w_{53} = w_{53} + \eta\delta_5 Out_3$$

Weights updating (6)



$$w_{64} = w_{64} + \eta\delta_6 Out_4$$

$$w_{65} = w_{65} + \eta\delta_6 Out_5$$

Weights Initialization

- Usually, the weights are initialized with small random values
- If the weights have large initial values
  - The sigmoid functions will reach saturation state soon
  - The system will clog at a local minimum or at a very flat plateau near the starting point
- Recomend for w_{ab}^{0} (link from neuron b to neuron a)
  - Let n_a be the number of neurons in the same layer as neuron a

$$w_{ab}^{0} \in [-\frac{1}{n_a}, \frac{1}{n_a}]$$

  - Let k_a be the number of neurons with forward connections to neuron a (=number of input connections of neuron a)

$$w_{ab}^{0} \in [-\frac{3}{\sqrt{k_a}}, \frac{3}{\sqrt{k_a}}]$$

Learning rate

- Important influence on the efficiency and convergence of the BP learning algorithm
  - A large value of η can accelerate the convergence of the learning process, but can cause the system to miss the global optimum or fall into the local optimum.
  - A small value of η can make the learning process take a very long time
- Usually be chosen experimentally for each problem
- Good values of learning rate at the beginning may not be good at a later time
  - An adaptive (dynamic) learning rate should be used
- After updating the weights, check whether updating the weights reduces the error value

$$\Delta\eta = \begin{cases} a & \text{, if } \Delta E < 0 \text{ consistently} \\ -b\eta & \text{, if } \Delta E > 0 \\ 0 & \text{, otherwise.} \end{cases} \qquad (a, b > 0)$$